I was just on LinkedIn and it didn't seem like I had 200 notifications lol

Then with Facebook it's going down but I haven't I didn't post yesterday really like I have been I posted once

I'm not emotional reacting to it. I'm, I don't know if you've noticed, but like, I've just been trying to understand it, and that's why I talk to you about it every day. Active daily, you know, not pivoting, but articulation.

My goal is to create my, like my system which can already produce artifacts for each of these simultaneously, because I want to keep producing a multi-channel output. I just want it to be more efficient as I go along, because like, eventually, here soon, like my system already produces artifacts, like, I can just start, you know, using my system, which I haven't really yet.

Um, can you give me, like, an anti-gravity prompt to ask it how close I am to actually being able to use it for, um, same multi-channel presence with a lower cognitive load from manual distribution to system-distributed signal?

📦

```
ANTI_GRAVITY_MODE: READINESS_AUDIT


Objective:

Evaluate how close the current system architecture is to reliably producing

multi-channel signal (LinkedIn, Facebook, YouTube, Substack, X, Reddit)
```

from a single core artifact with reduced cognitive load compared to manual distribution.

Scope:

Assess structural readiness — not motivation, not aspiration.

Audit Across These Dimensions:

1. Core Artifact Integrity
  - Is there a stable single-source-of-truth document format?
  - Is it consistently structured (sections, thesis, spine clarity)?
  - Is it machine-transformable without ambiguity?

2. Transformation Layer
  - Do deterministic templates exist per channel?
  - Are wrappers formalized (tone, length, formatting constraints)?
  - Is channel-specific compression logic defined?

3. Automation Readiness
  - Can artifact → derivative outputs occur without manual rewriting?
  - Are generation steps reproducible?
  - Are file paths / naming conventions standardized?

4. Cognitive Load Delta
  - Estimate manual distribution load (time + context switching).
  - Estimate system-distributed load.
  - Quantify reduction as percentage.

```
5. Failure Points
  - Where would coherence drift occur?
  - Where would overfitting to one platform distort the spine?
  - Where is manual intervention still required?

6. Operational Threshold
  - Is the system at:
      0–40%: Conceptual
      40–70%: Semi-operational
      70–90%: Operational with guardrails
      90–100%: Fully system-distributed
  - Justify rating.

Output Format:
- Structural score (percentage)
- Top 3 blockers
- Fastest path to operational threshold (minimal steps)
- Risk assessment if activated immediately

Constraint:
Respond in systems-analysis format only.
No motivational language.
No speculation beyond observable architecture.
```

Because, like, I've already made a compiler and everything. Do you remember everything we built?

I guess one of the next things I really want though is my HQ app to have a dashboard that I can control everything from because just looking at terminals and executing that way is not the best for me but we can make a dashboard so it's easier right? I mean that's what already what we kind of started doing with the HQ app

```
E:\signal_agent\
```

“Antigravity is saying I can't use Gemini 3.1 Pro because it's not available on the version. Please upgrade to the latest version. How do I upgrade my Antigravity?”

00:11

“No, no, it literally is anti-gravity through freaking um meta.”

00:05

“Well, I'm at Google Antigravity. It's Google, not Meta.”

00:03

This is what I'm using, but I don't wanna, it says download for Windows. I already have it downloaded on Windows. How would I be able to upgrade it, you think?

Thought for 14s

`--force`

✅

```
winget install --id Google.Antigravity --force
```

Okay, give me a prompt for Antigravity to check for an update. I'll use a Gemini 3.0, which mine has, so it should be able to, you know, walk through and do it.

```
SYSTEM TASK: IDE Update & Model Availability Audit


Objective:

Determine whether this installation of Google Antigravity is running the latest Windows version and whether Gemini 3.1 Pro should be available on this build.


Steps:


1. Identify current Antigravity version.

  - Provide exact version number.
```

```
  - Provide release channel (stable / preview / beta if applicable).


2. Compare against latest available Windows release.
  - Query official release metadata.
  - Report if this installation is outdated.


3. If outdated:
  - Provide exact upgrade method for Windows.
  - Specify whether in-app update exists.
  - If not, provide official installer method.
  - If applicable, provide winget upgrade command.


4. If up to date but Gemini 3.1 Pro unavailable:
  - Determine if model access is:
       a) Version-gated
       b) Account-gated
       c) Region-gated
       d) Feature-flag rollout limited
  - Explain which is most likely and why.


5. Output Format:
  - Current Version:
  - Latest Version:
  - Upgrade Required: YES / NO
  - Model Access Status:
  - Required Action Steps (minimal, ordered list)


Constraint:
```

```
Do not provide marketing copy.

Respond with structured diagnostic output only.
```

⚠️

LinkedIn Social Offload Deliverables

1. File Contents

domain_packs/linkedin_pack.yaml

```yaml
scope: "domain:linkedin"
pack_version: "1.0.0"
created_at: "2026-02-20T15:00:00Z"
boundary_conditions:
  max_word_count: 250
  min_word_count: 20
  max_claims: 4
disallowed_phrases:
  - "game changer"
  - "synergy"
  - "delve"
  - "crushing it"
  - "thought leader"
  - "🚀"
```

```
  - "🔥"
allowed_output_classes: []
required_invariants: []
constraint_rules:
  - constraint_id: LINKEDIN_ALLOW_OFFLOAD
    rule_type: ALLOW
    trigger:
      capability_id: "content:text"
    predicate: {}
  - constraint_id: LINKEDIN_DENY_EXCESS_CLAIMS
    rule_type: DENY
    trigger:
      capability_id: "content:text"
    predicate:
      op: "GT"
      left: "snapshot.metrics.claims_count"
      right: 4
```

templates/linkedin_post.jinja

jinja

```
{% set hook_text = sections.get('hook') or sections.get('preamble') or 'Untitled Update'
%}
{{ hook_text }}
{% if sections.get('body') %}
{{ sections.get('body') }}
{% else %}
{% for text in sections.values() %}
{% if loop.index > 1 %}{{ text }}
{% endif %}
{% endfor %}
{% endif %}
{% if claims %}
{% for claim in claims %}
• {{ claim }}
{% endfor %}
{% endif %}
{% if sections.get('conclusion') %}
{{ sections.get('conclusion') }}
{% endif %}
```

social_offload.py

python

```
import argparse
import hashlib
import json
import logging
import os
import sys
import time
from pathlib import Path
sys.path.insert(0, str(Path(__file__).resolve().parent.parent.parent.parent))
from jinja2 import Environment, FileSystemLoader
from app.utils.reprojection import reproject_checkpoint, extract_artifact_state
from app.utils.exceptions import ConstraintViolation
logging.basicConfig(level=logging.INFO, format="%(asctime)s [%(levelname)s]
%(message)s")
logger = logging.getLogger(__name__)
def sha256_hash(data: str) -> str:
    return hashlib.sha256(data.encode("utf-8")).hexdigest()
def render_linkedin_output(state, template_dir: Path) -> str:
    env = Environment(loader=FileSystemLoader(str(template_dir)), trim_blocks=True,
lstrip_blocks=True)
    template = env.get_template("linkedin_post.jinja")
    rendered = template.render(sections=state.sections, claims=state.claims,
word_count=state.word_count)
    return rendered.strip()
def run_social_offload(artifact_path: Path, channel: str, out_dir: Path, pack_path: Path,
dry_run: bool):
    logger.info(f"Starting {channel} social offload for {artifact_path}")

    try:
        with open(artifact_path, "r", encoding="utf-8") as f:
            content = f.read()
    except FileNotFoundError:
        logger.error(f"Artifact not found: {artifact_path}")
        sys.exit(1)

    state = extract_artifact_state(content)
    artifact_id = sha256_hash(content)[:12]

    pack_version = "1.0.0"
    template_version = "1.0.0"
```

```
    render_id_input = f"{artifact_id}_{pack_version}_{template_version}"
    render_id = sha256_hash(render_id_input)[:12]

    logger.info(f"Artifact ID: {artifact_id} | Render ID: {render_id}")

    try:
        report = reproject_checkpoint(
            artifact=content,
            pack_path=str(pack_path),
            execution_context_id=render_id,
            log_dir=out_dir.parent / "logs" / "reprojection"
        )
    except ConstraintViolation as e:
        logger.error(f"Failing closed due to constraint violation: {e}")
        write_fault_report(out_dir.parent / "logs", artifact_path, artifact_id, channel,
pack_version, template_version, render_id, str(e))
        sys.exit(1)

    template_dir = Path(__file__).resolve().parent / "templates"
    try:
        rendered_text = render_linkedin_output(state, template_dir)
    except Exception as e:
        logger.error(f"Template rendering failed: {e}")
        write_fault_report(out_dir.parent / "logs", artifact_path, artifact_id, channel,
pack_version, template_version, render_id, f"Template Error: {e}")
        sys.exit(1)

    if not dry_run:
        channel_out_dir = out_dir / channel / artifact_id
        channel_out_dir.mkdir(parents=True, exist_ok=True)
        out_file = channel_out_dir / f"{render_id}.txt"
        with open(out_file, "w", encoding="utf-8") as f:
            f.write(rendered_text)
        logger.info(f"Successfully wrote output to {out_file}")

    log_entry = {
        "timestamp": time.time(),
        "artifact_path": str(artifact_path),
        "artifact_id": artifact_id,
        "channel": channel,
```

```
        "pack_version": pack_version,
        "template_version": template_version,
        "render_id": render_id,
        "status": "ok",
        "word_count": state.word_count,
        "claim_count": len(state.claims)
    }
    log_dir = out_dir.parent / "logs"
    log_dir.mkdir(parents=True, exist_ok=True)
    with open(log_dir / "social_offload_runs.jsonl", "a", encoding="utf-8") as f:
        f.write(json.dumps(log_entry) + "\n")

    logger.info("Pipeline completed successfully.")
def write_fault_report(log_dir: Path, artifact_path, artifact_id, channel, pack_version,
template_version, render_id, fault_details):
    log_entry = {
        "timestamp": time.time(),
        "artifact_path": str(artifact_path),
        "artifact_id": artifact_id,
        "channel": channel,
        "pack_version": pack_version,
        "template_version": template_version,
        "render_id": render_id,
        "status": "fault",
        "fault_codes": fault_details,
    }
    log_dir.mkdir(parents=True, exist_ok=True)
    with open(log_dir / "social_offload_runs.jsonl", "a", encoding="utf-8") as f:
        f.write(json.dumps(log_entry) + "\n")
if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--artifact", required=True, type=Path)
    parser.add_argument("--channel", required=True, choices=["linkedin"])
    parser.add_argument("--out", type=Path, default=Path("data/social_offload/outputs"))
    parser.add_argument("--pack", type=Path, required=True)
    parser.add_argument("--dry-run", action="store_true")
    args = parser.parse_args()
    run_social_offload(args.artifact, args.channel, args.out, args.pack, args.dry_run)
core_artifact_fixture.md (Passing Test)
markdown
```

hook
Systems fail because complexity outpaces observability.
body
When building deterministic systems, the biggest risk isn't network failure. It's coherence drift.
We addressed this by formalizing robust constraint packs and decoupling generation from projection.
This ensures output stays rigidly bounded without needing a human in the loop.
key_takeaways
- Determinism requires hashes, not just IDs.
- Constraints must fail-closed.
- Observability is non-negotiable.
conclusion
Keep your systems rigid so your agents can be flexible.
expected_linkedin_output.txt (Golden Snapshot)
text
Systems fail because complexity outpaces observability.
When building deterministic systems, the biggest risk isn't network failure. It's coherence drift.
We addressed this by formalizing robust constraint packs and decoupling generation from projection.
This ensures output stays rigidly bounded without needing a human in the loop.
• Determinism requires hashes, not just IDs.
• Constraints must fail-closed.
• Observability is non-negotiable.
Keep your systems rigid so your agents can be flexible.
core_artifact_failing.md
 (Failing Test)
markdown
hook
This is a total game changer for the industry! 🚀
body
We are crushing it right now. Our synergy is off the charts.
This new platform will revolutionize the way we work. Delve into the details!
claims
- Claim 1
- Claim 2
- Claim 3
- Claim 4
- Claim 5

- Claim 6
Expected Fault JSON (from
social_offload_runs.jsonl
)
json
{"timestamp": 1771619702.8367202, "artifact_path": "e:\\signal_agent\\tests\\fixtures\\social_offload\\core_artifact_failing.md", "artifact_id": "f16843e8b19f", "channel": "linkedin", "pack_version": "1.0.0", "template_version": "1.0.0", "render_id": "99aee2d83dc5", "status": "fault", "fault_codes": "Constraint violation in e:\\signal_agent\\constraints\\packs\\domain\\linkedin_pack.yaml"}
2. Integration Notes
Placement in Repo
Constraint Pack:
…\constraints\packs\domain\linkedin_pack.yaml
Runner Script:
…\app\agents\social_offload\social_offload.py
Jinja Template:
…\social_offload\templates\linkedin_post.jinja
Test Fixtures:
…\fixtures\social_offload\core_artifact.md
 and
core_artifact_failing.md
Exact CLI Commands
To execute the pipeline and write the output file deterministically:

powershell
Run the pipeline against a core artifact
python e:\signal_agent\app\agents\social_offload\social_offload.py --artifact e:\signal_agent\tests\fixtures\social_offload\core_artifact.md --channel linkedin --pack e:\signal_agent\constraints\packs\domain\linkedin_pack.yaml
To run and trigger a constraint failure:

powershell
Run the pipeline against a non-compliant artifact
python e:\signal_agent\app\agents\social_offload\social_offload.py --artifact e:\signal_agent\tests\fixtures\social_offload\core_artifact_failing.md --channel linkedin --pack e:\signal_agent\constraints\packs\domain\linkedin_pack.yaml
Expected Output Tree
After running both commands, the tree expands to include the output paths and the logs directory containing the JSONL records:

```text
data/social_offload/
├── logs/
│   ├── reprojection/
│   │   ├── ad74d7cddb98.jsonl  # Passing reprojection audit event
│   │   └── 99aee2d83dc5.jsonl  # Failing reprojection audit event
│   └── social_offload_runs.jsonl # Complete pipeline fault/success audit
└── outputs/
    └── linkedin/
        └── <artifact_id>/            # (e.g. 6492300701b9)
            └── <render_id>.txt         # (e.g. ad74d7cddb98.txt)
```

Thought for 18s

`linkedin_pack.yaml`
`linkedin_post.jinja`

```
data/social_offload/logs/social_offload_runs.jsonl
data/social_offload/outputs/linkedin/<artifact_id>/
```

```
POST /api/run/social_offload
GET /api/runs?limit=50
GET /api/outputs?channel=linkedin&artifact_id=...
```

📦

```
# From: E:\signal_agent
python -m venv .venv
.\.venv\Scripts\pip install fastapi uvicorn pydantic

# Run (no file creation yet—this just verifies deps)
.\.venv\Scripts\python -c "import fastapi,uvicorn; print('ok')"
# Save as: E:\signal_agent\app\hq\dashboard\hq_dashboard.py
from fastapi import FastAPI
from pydantic import BaseModel
import subprocess
from pathlib import Path
```

```
import json


APP_ROOT = Path(r"E:\signal_agent")
SOCIAL_OFFLOAD = APP_ROOT / "app" / "agents" / "social_offload" /
"social_offload.py"
RUNS_JSONL = APP_ROOT / "data" / "social_offload" / "logs" /
"social_offload_runs.jsonl"
OUTPUTS_DIR = APP_ROOT / "data" / "social_offload" / "outputs"


app = FastAPI(title="HQ Dashboard API")


class SocialOffloadReq(BaseModel):
   artifact: str
   channel: str = "linkedin"
   pack: str
   dry_run: bool = False


@app.post("/api/run/social_offload")
def run_social_offload(req: SocialOffloadReq):
   cmd = [
       "python",
       str(SOCIAL_OFFLOAD),
       "--artifact", req.artifact,
       "--channel", req.channel,
       "--pack", req.pack,
   ]
   if req.dry_run:
       cmd.append("--dry-run")
```

```
    p = subprocess.run(cmd, cwd=str(APP_ROOT), capture_output=True,
text=True)
    return {
        "returncode": p.returncode,
        "stdout": p.stdout[-4000:],   # tail to keep response small
        "stderr": p.stderr[-4000:],
        "cmd": cmd,
    }

@app.get("/api/runs")
def get_runs(limit: int = 50):
    if not RUNS_JSONL.exists():
        return {"runs": []}
    lines = RUNS_JSONL.read_text(encoding="utf-8").splitlines()
    tail = lines[-limit:]
    runs = [json.loads(x) for x in tail if x.strip()]
    return {"runs": runs[::-1]}  # newest first

@app.get("/api/outputs")
def list_outputs(channel: str = "linkedin", artifact_id: str | None =
None):
    base = OUTPUTS_DIR / channel
    if not base.exists():
        return {"paths": []}
    if artifact_id:
        base = base / artifact_id
    if not base.exists():
        return {"paths": []}
```

```
    return {"paths": [str(p) for p in sorted(base.glob("**/*")) if
p.is_file()]}

# Run the API

.\.venv\Scripts\python -m uvicorn app.hq.dashboard.hq_dashboard:app --host
127.0.0.1 --port 8787
```

```
http://127.0.0.1:8787/docs
```

As much as I love the idea of building it for two hours in my PowerShell, how about you just give me a prompt for anti-gravity to start building it for me?

📦

```
TASK: Build HQ Dashboard Control Surface for Social Offload


Objective:

Create a minimal, deterministic HQ dashboard that wraps the existing

social_offload pipeline without modifying core logic.


Constraints:

- DO NOT rewrite or duplicate social_offload.py logic.

- UI must only call existing CLI or module functions.

- Preserve fail-closed behavior.

- Preserve JSONL audit logging.

- No marketing text. No speculative features.


Scope Phase 1:

LinkedIn channel only.


Architecture Requirements:
```

```
1. Backend
  - Use FastAPI.
  - Location: app/hq/dashboard/
  - Expose endpoints:

    POST /api/run/social_offload
      - Inputs: artifact path, pack path, dry_run
      - Executes existing social_offload.py via subprocess
      - Returns: returncode, stdout tail, stderr tail

    GET /api/runs
      - Reads data/social_offload/logs/social_offload_runs.jsonl
      - Returns last N entries (default 50)

    GET /api/outputs
      - Lists files under data/social_offload/outputs/linkedin/

2. Frontend (Minimal)
  - Serve a single HTML page from FastAPI.
  - Page includes:
      - Artifact path input
      - Pack path input
      - Dry-run toggle
      - "Execute" button
      - Output preview pane
      - Recent runs table
  - No external frameworks.
```

```
  - Vanilla HTML + minimal JS fetch calls.


3. Determinism Rules
  - All paths configurable at top of file.
  - No hard-coded absolute Windows paths.
  - Derive repo root relative to file location.
  - No dynamic model calls.


4. Deliverables
  - hq_dashboard.py
  - index.html (embedded or template)
  - requirements.txt (FastAPI + uvicorn only)
  - Run command example (PowerShell)


5. Verification
  - Provide exact CLI command to start dashboard.
  - Confirm that running a passing artifact produces:
      - JSONL audit entry
      - Output file under outputs/linkedin/<artifact_id>/
  - Confirm that failing artifact logs fault and does not write output
file.


Output Format:
- File tree
- Full code blocks per file
- PowerShell run instructions
- No explanation outside technical notes.
```

HQ Dashboard Control Surface

File Tree

```
e:\signal_agent\app\hq\dashboard\
├── hq_dashboard.py
├── index.html
└── requirements.txt
```

Source Code

app/hq/dashboard/requirements.txt

text

```
fastapi>=0.111.0
uvicorn>=0.30.0
```

app/hq/dashboard/index.html

html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>HQ Dashboard - Social Offload</title>
    <style>
        body { font-family: system-ui, -apple-system, sans-serif; margin: 0; padding: 20px;
background: #f4f4f5; color: #18181b; }
        .container { max-width: 1200px; margin: 0 auto; display: grid; gap: 20px;
grid-template-columns: 350px 1fr; }
        .card { background: white; padding: 20px; border-radius: 8px; box-shadow: 0 1px
3px rgba(0,0,0,0.1); }
        h1, h2, h3 { margin-top: 0; }
        .form-group { margin-bottom: 15px; }
        label { display: block; font-weight: 500; margin-bottom: 5px; font-size: 14px; }
        input[type="text"] { width: 100%; padding: 8px; border: 1px solid #d4d4d8;
border-radius: 4px; box-sizing: border-box; }
        .toggle { display: flex; align-items: center; gap: 8px; font-size: 14px; }
        button { background: #09090b; color: white; border: none; padding: 10px 15px;
border-radius: 4px; cursor: pointer; font-weight: 500; width: 100%; }
        button:hover { background: #27272a; }
        button:disabled { background: #a1a1aa; cursor: not-allowed; }
        pre { background: #18181b; color: #10b981; padding: 15px; border-radius: 4px;
overflow-x: auto; font-size: 13px; min-height: 100px; margin: 0; white-space: pre-wrap;
word-wrap: break-word;}
        table { width: 100%; border-collapse: collapse; font-size: 13px; }
        th, td { padding: 10px; text-align: left; border-bottom: 1px solid #e4e4e7; }
        th { font-weight: 600; color: #52525b; }
        .status-ok { color: #10b981; font-weight: 600;}
        .status-fault { color: #ef4444; font-weight: 600;}
        .full-width { grid-column: 1 / -1; }
        .outputs-list { list-style: none; padding: 0; margin: 0; font-size: 13px;}
        .outputs-list li { padding: 5px 0; border-bottom: 1px solid #f4f4f5; }
    </style>
```

```
</head>
<body>
    <div class="container">
        <div class="card full-width">
            <h1>HQ Social Offload Dashboard</h1>
        </div>

        <div class="card">
            <h2>Execute Pipeline</h2>
            <form id="execute-form">
                <div class="form-group">
                    <label for="artifact">Core Artifact Path</label>
                    <input type="text" id="artifact" name="artifact" required
value="tests/fixtures/social_offload/core_artifact.md">
                </div>
                <div class="form-group">
                    <label for="pack">Constraint Pack Path</label>
                    <input type="text" id="pack" name="pack" required
value="constraints/packs/domain/linkedin_pack.yaml">
                </div>
                <div class="form-group toggle">
                    <input type="checkbox" id="dry-run" name="dry_run">
                    <label for="dry-run" style="margin:0">Dry Run (Do not write file)</label>
                </div>
                <button type="submit" id="submit-btn">Run Social Offload
(LinkedIn)</button>
            </form>
        </div>
        <div class="card">
            <h2>Execution Output</h2>
            <pre id="output-pane">Ready...</pre>
        </div>
        <div class="card full-width">
            <h2>Recent Runs (Audit Log)</h2>
            <table id="runs-table">
                <thead>
                    <tr>
                        <th>Timestamp</th>
                        <th>Status</th>
                        <th>Artifact ID</th>
```

```
                <th>Render ID</th>
                <th>Details / Faults</th>
            </tr>
        </thead>
        <tbody>
            <tr><td colspan="5">Loading...</td></tr>
        </tbody>
    </table>
</div>
<div class="card full-width">
    <h2>Generated Output Files</h2>
    <ul class="outputs-list" id="outputs-list">
        <li>Loading...</li>
    </ul>
</div>
</div>
<script>
    async function fetchRuns() {
        try {
            const res = await fetch('/api/runs');
            const data = await res.json();
            const tbody = document.querySelector('#runs-table tbody');
            tbody.innerHTML = '';
            if (!data || data.length === 0) {
                tbody.innerHTML = '<tr><td colspan="5">No runs found</td></tr>';
                return;
            }
            data.forEach(run => {
                const statusClass = run.status === 'ok' ? 'status-ok' : 'status-fault';
                const date = new Date(run.timestamp * 1000).toLocaleString();
                let details = '-';
                if (run.status === 'fault') {
                    details = run.fault_codes || 'Unknown Fault';
                } else if (run.status === 'ok') {
                    details = `Words: ${run.word_count}, Claims: ${run.claim_count}`;
                }
                const tr = document.createElement('tr');
                tr.innerHTML =
                    <td>${date}</td>
                    <td class="${statusClass}">${run.status.toUpperCase()}</td>
```

```
                <td><code style="background:#f4f4f5;padding:2px
4px;border-radius:3px">${run.artifact_id || '-'}</code></td>
                <td><code style="background:#f4f4f5;padding:2px
4px;border-radius:3px">${run.render_id || '-'}</code></td>
                <td>${details}</td>
            `;
            tbody.appendChild(tr);
        });
    } catch (err) {
        console.error(err);
    }
}
async function fetchOutputs() {
    try {
        const res = await fetch('/api/outputs');
        const data = await res.json();
        const ul = document.getElementById('outputs-list');
        ul.innerHTML = '';
        if (!data.files || data.files.length === 0) {
            ul.innerHTML = '<li>No files generated yet</li>';
            return;
        }
        data.files.forEach(f => {
            const li = document.createElement('li');
            li.innerHTML = `<code style="background:#f4f4f5;padding:2px
4px;border-radius:3px;font-family:monospace">${f}</code>`;
            ul.appendChild(li);
        });
    } catch (err) {
        console.error(err);
    }
}
document.getElementById('execute-form').addEventListener('submit', async (e) =>
{
    e.preventDefault();
    const btn = document.getElementById('submit-btn');
    const outPane = document.getElementById('output-pane');
    btn.disabled = true;
    btn.textContent = 'Running...';
    outPane.textContent = 'Executing subprocess...';
```

```
        const payload = {
            artifact_path: document.getElementById('artifact').value,
            pack_path: document.getElementById('pack').value,
            dry_run: document.getElementById('dry-run').checked
        };
        try {
            const res = await fetch('/api/run/social_offload', {
                method: 'POST',
                headers: {'Content-Type': 'application/json'},
                body: JSON.stringify(payload)
            });
            const result = await res.json();

            let outText = `Return Code: ${result.returncode}\n\n`;
            if (result.stdout_tail) outText += `--- STDOUT
---\n${result.stdout_tail}\n`;
            if (result.stderr_tail) outText += `--- STDERR ---\n${result.stderr_tail}\n`;
            outPane.textContent = outText;
        } catch (err) {
            outPane.textContent = `Error: ${err.message}`;
        } finally {
            btn.disabled = false;
            btn.textContent = 'Run Social Offload (LinkedIn)';
            fetchRuns();
            fetchOutputs();
        }
    });
    // Init
    fetchRuns();
    fetchOutputs();
  </script>
</body>
</html>
```

app/hq/dashboard/hq_dashboard.py

python

```
import json
import logging
import subprocess
import sys
from collections import deque
```

```
from pathlib import Path
from pydantic import BaseModel
from fastapi import FastAPI
from fastapi.responses import HTMLResponse
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)
# Derive paths relative to this file to ensure determinism and portability
DASHBOARD_DIR = Path(__file__).resolve().parent
REPO_ROOT = DASHBOARD_DIR.parent.parent.parent
SOCIAL_OFFLOAD_SCRIPT = REPO_ROOT / "app" / "agents" / "social_offload" /
"social_offload.py"
LOGS_JSONL = REPO_ROOT / "data" / "social_offload" / "logs" /
"social_offload_runs.jsonl"
OUTPUTS_DIR = REPO_ROOT / "data" / "social_offload" / "outputs" / "linkedin"
app = FastAPI(title="HQ Social Offload Dashboard")
class RunRequest(BaseModel):
    artifact_path: str
    pack_path: str
    dry_run: bool = False
@app.get("/", response_class=HTMLResponse)
async def serve_dashboard():
    index_path = DASHBOARD_DIR / "index.html"
    try:
        with open(index_path, "r", encoding="utf-8") as f:
            return f.read()
    except FileNotFoundError:
        return "<h1>Error: index.html not found</h1>"
@app.post("/api/run/social_offload")
async def run_social_offload(req: RunRequest):
    # Resolve absolute paths based on REPO_ROOT
    abs_artifact = (REPO_ROOT / req.artifact_path).resolve()
    abs_pack = (REPO_ROOT / req.pack_path).resolve()

    cmd = [
        sys.executable,
        str(SOCIAL_OFFLOAD_SCRIPT),
        "--artifact", str(abs_artifact),
        "--channel", "linkedin",
        "--pack", str(abs_pack)
    ]
```

```
    if req.dry_run:
        cmd.append("--dry-run")

    logger.info(f"Executing: {' '.join(cmd)}")

    proc = subprocess.Popen(
        cmd,
        cwd=str(REPO_ROOT),
        stdout=subprocess.PIPE,
        stderr=subprocess.PIPE,
        text=True
    )

    stdout, stderr = proc.communicate()

    # Return last 2000 chars of tail to prevent giant payload
    return {
        "returncode": proc.returncode,
        "stdout_tail": stdout[-2000:] if stdout else "",
        "stderr_tail": stderr[-2000:] if stderr else "",
    }
@app.get("/api/runs")
async def get_runs(limit: int = 50):
    if not LOGS_JSONL.exists():
        return []

    # Tail the JSONL file
    lines = deque(maxlen=limit)
    try:
        with open(LOGS_JSONL, "r", encoding="utf-8") as f:
            for line in f:
                line = line.strip()
                if line:
                    lines.append(line)
    except Exception as e:
        logger.error(f"Error reading logs: {e}")
        return []

    results = []
    # Reverse so newest is first
```

```
    for line in reversed(list(lines)):
        try:
            results.append(json.loads(line))
        except json.JSONDecodeError:
            pass

    return results
@app.get("/api/outputs")
async def get_outputs():
    if not OUTPUTS_DIR.exists():
        return {"files": []}

    files = []
    try:
        # Recursively find generated txt files
        for p in OUTPUTS_DIR.rglob("*.txt"):
            rel_path = p.relative_to(REPO_ROOT)
            files.append(str(rel_path).replace("\\", "/"))
    except Exception as e:
        logger.error(f"Error reading outputs: {e}")

    return {"files": sorted(files, reverse=True)}
```

Run Instructions (PowerShell)

1. Install Dependencies

powershell

```
pip install -r e:\signal_agent\app\hq\dashboard\requirements.txt
```

2. Start the Dashboard Backend

powershell

```
$env:PYTHONPATH = "e:\signal_agent"
cd e:\signal_agent
uvicorn app.hq.dashboard.hq_dashboard:app --host 127.0.0.1 --port 8000
```

3. Open the Dashboard UI

Open http://localhost:8000 in your web browser.

Verification Notes

The dashboard calls the preexisting generic runner via Python subprocess.Popen rather than duplicating any internal pipeline implementations.

It relies entirely on the same underlying reprojection_checkpoint constraints and Jinja templates as the backend CLI.

Execution events (both constraints PASS writing successfully to outputs/linkedin/ and instances where constraints FAIL emitting a JSON record with fault reasons) reflect faithfully within the "Recent Runs (Audit Log)" HTML dashboard table utilizing GET /api/runs.

✅ `social_offload.py`

✅
✅ `sys.executable`
✅
✅

`DASHBOARD_DIR.parent.parent.parent` `app/hq/dashboard` `.../app` `.../signal_agent`

`E:\signal_agent\app\hq\dashboard\hq_dashboard.py`
`parent` `dashboard`
`parent.parent` `hq`
`parent.parent.parent` `app`
`REPO_ROOT` `E:\signal_agent\app`
`REPO_ROOT / "app" / "agents" / ...`
`E:\signal_agent\app\app\agents\...`
`E:\signal_agent` `dashboard`

✅

```
REPO_ROOT = DASHBOARD_DIR.parent.parent.parent
```

```
REPO_ROOT = DASHBOARD_DIR.parents[3]  # .../signal_agent
```

```
data/social_offload/outputs/linkedin/<artifact_id>/<render_id>.txt
```

```
OUTPUTS_DIR = REPO_ROOT / "data" / "social_offload" / "outputs" /
"linkedin"
```

`*.txt`

`/api/runs`

`[]`

```
if (!data || data.length === 0)
```

```
abs_artifact = (REPO_ROOT / req.artifact_path).resolve()
```

`REPO_ROOT\C:\...`

```
artifact_in = Path(req.artifact_path)
pack_in = Path(req.pack_path)

abs_artifact = artifact_in if artifact_in.is_absolute() else (REPO_ROOT /
artifact_in).resolve()
```

```
abs_pack = pack_in if pack_in.is_absolute() else (REPO_ROOT /
pack_in).resolve()
```

`PYTHONPATH` `REPO_ROOT`

📦

```
# PATCH 1: Fix REPO_ROOT
REPO_ROOT = DASHBOARD_DIR.parents[3]  # E:\signal_agent


# PATCH 2: Support absolute paths
artifact_in = Path(req.artifact_path)
pack_in = Path(req.pack_path)
abs_artifact = artifact_in if artifact_in.is_absolute() else (REPO_ROOT /
artifact_in).resolve()
abs_pack = pack_in if pack_in.is_absolute() else (REPO_ROOT /
pack_in).resolve()
```

`data/social_offload/logs/social_offload_runs.jsonl`
`data/social_offload/outputs/linkedin/...`

`tests/fixtures/social_offload/core_artifact_failing.md`

`status=fault`

📦

```
Apply two fixes to app/hq/dashboard/hq_dashboard.py:


1) REPO_ROOT is incorrect. DASHBOARD_DIR is .../app/hq/dashboard. Set
REPO_ROOT to the repo root (E:\signal_agent) by using
DASHBOARD_DIR.parents[3]. Ensure SOCIAL_OFFLOAD_SCRIPT resolves to
E:\signal_agent\app\agents\social_offload\social_offload.py (not
...\app\app\...).


2) Support absolute paths in RunRequest. If req.artifact_path or
req.pack_path is absolute, use it as-is; otherwise resolve relative to
REPO_ROOT.


Return the updated hq_dashboard.py only. No other file changes.
```

As much as I love the idea of building it for two hours in my PowerShell, how about you just give me a prompt for anti-gravity to start building it for me?

📦

```
TASK: Construct HQ Dashboard Control Surface (Phase 1 - LinkedIn Only)


MISSION:

Build a minimal, deterministic dashboard that wraps the existing

social_offload pipeline without modifying its internal logic.


NON-NEGOTIABLE CONSTRAINTS:

- Do NOT rewrite social_offload.py.

- Do NOT duplicate constraint logic.

- Dashboard must call existing pipeline via subprocess.

- Fail-closed behavior must remain intact.

- JSONL audit logging must remain untouched.

- No speculative features. No marketing language.


REPO CONTEXT:

Root directory = signal_agent/

Existing runner = app/agents/social_offload/social_offload.py

Constraint pack = constraints/packs/domain/linkedin_pack.yaml

Logs = data/social_offload/logs/social_offload_runs.jsonl

Outputs = data/social_offload/outputs/linkedin/
```

```
DELIVERABLE STRUCTURE:

Create:


app/hq/dashboard/

   hq_dashboard.py

   index.html

   requirements.txt


ARCHITECTURE REQUIREMENTS:


1) Path Determinism

  - Derive REPO_ROOT relative to hq_dashboard.py using
DASHBOARD_DIR.parents[3]

  - Do NOT hardcode absolute Windows paths

  - Support both relative and absolute artifact paths


2) Backend (FastAPI)

  Expose endpoints:


  GET /

     → Serve index.html


  POST /api/run/social_offload

     → Accept: artifact_path, pack_path, dry_run

     → Execute social_offload.py via sys.executable

     → Return: returncode, stdout_tail (<=2000 chars), stderr_tail (<=2000
chars)


  GET /api/runs
```

```
    → Tail last 50 entries from social_offload_runs.jsonl
    → Return parsed JSON array (newest first)

  GET /api/outputs
    → Recursively list *.txt under outputs/linkedin/
    → Return relative paths

3) Frontend (Single HTML file)
  - Artifact path input
  - Pack path input
  - Dry-run toggle
  - Execute button
  - Output preview pane
  - Recent runs table (status color-coded)
  - Output files list

  No external frameworks.
  Vanilla JS fetch only.

4) Verification Requirements
  After generating files, provide:

  - Exact PowerShell commands to:
      pip install -r requirements.txt
      set PYTHONPATH
      start uvicorn server

  - Explicit test steps:
```

```
       Run passing fixture

       Run failing fixture

       Confirm:

           PASS → writes output + JSONL entry

           FAIL → JSONL fault entry only (no output file)


5) Output Format

  Return:


  - File tree

  - Full contents of each file

  - PowerShell run commands

  - Verification checklist


No commentary outside structured technical output.
```

Verification Checklist

Execute

tests/fixtures/social_offload/core_artifact.md via UI.

Observe return code 0 in execution output window.

Verify identical OK entry appears in "Recent Runs (Audit Log)" top row.

Verify written linkedin/<artifact_id>/<render_id>.txt appears in "Generated Output Files" window.

Execute

tests/fixtures/social_offload/core_artifact_failing.md via UI.

Observe non-zero return code (e.g. 1) in execution output window, containing constraint violation STDERR inline.

Verify top row in "Recent Runs (Audit Log)" updates with color-coded FAULT state matching the latest failing request.

Verify "Generated Output Files" count does not increment for failed run.

`data/social_offload/logs/social_offload_runs.jsonl`

`artifact_path`

`status` `ok` `fault`

`data/social_offload/outputs/linkedin/`

`...\app\app\agents\...`

📦

```
cd e:\signal_agent
python -c "from pathlib import Path;
p=Path(r'app\hq\dashboard\hq_dashboard.py').resolve(); d=p.parent;
print('DASHBOARD_DIR=',d); print('REPO_ROOT=',d.parents[3]);
print('SOCIAL_OFFLOAD=',d.parents[3]/'app'/'agents'/'social_offload'/'soci
al_offload.py')"
```

`REPO_ROOT` `e:\signal_agent`
`SOCIAL_OFFLOAD` `e:\signal_agent\app\agents\social_offload\social_offload.py`
`e:\signal_agent\app`

```
dry_run
```

```
stdout_tail
stderr_tail
```

Good — that checklist is the right behavioral contract. Now make it mechanically verifiable (so you're not trusting the UI's vibe).

Add these two hard checks (so "count does not increment" can't lie)

Confirm the log file appended

After each run, open:

data/social_offload/logs/social_offload_runs.jsonl

Verify the newest line matches the run you just did:

same artifact_path

same status (ok vs fault)

timestamp is current

Confirm the output directory delta

Before failing run, note newest output file timestamp under:

data/social_offload/outputs/linkedin/

After failing run, verify no new file was created (by timestamp), not just "list count."

That removes caching/UI refresh ambiguity.

One critical sanity check (because this can silently fail)

In your generated backend, REPO_ROOT must be correct or it'll try to run:
...\app\app\agents\... and you'll get weird "file not found" or import errors.

Quick deterministic check from PowerShell (no guessing):

📦 VERBATIM-SAFE COMMANDS

```
cd e:\signal_agent
python -c "from pathlib import Path;
p=Path(r'app\hq\dashboard\hq_dashboard.py').resolve(); d=p.parent;
print('DASHBOARD_DIR=',d); print('REPO_ROOT=',d.parents[3]);
print('SOCIAL_OFFLOAD=',d.parents[3]/'app'/'agents'/'social_offload'/'social_offload.py')
"
```

Expected:

REPO_ROOT prints e:\signal_agent

SOCIAL_OFFLOAD prints e:\signal_agent\app\agents\social_offload\social_offload.py

If it prints e:\signal_agent\app as repo root, fix that before testing (otherwise the UI can "run" but not actually execute the right script).

If your return code is 0 but you don't see files

That usually means:

dry_run was on

outputs path mismatch

UI list isn't refreshing / caching

pipeline wrote somewhere else due to CWD/PYTHONPATH issues

In that case, don't troubleshoot blindly — paste:

the stdout_tail

the stderr_tail

and the printed paths from the command above

And I'll tell you exactly which layer failed. add in all context needed for me to prompt this to my codex cli

📦

```
TASK: Add mechanically verifiable checks + a repo-root sanity check to the HQ Dashboard verification process, and output the exact PowerShell commands needed to run those checks.


CONTEXT:

Repo root: e:\signal_agent

Dashboard backend: e:\signal_agent\app\hq\dashboard\hq_dashboard.py

Dashboard UI: e:\signal_agent\app\hq\dashboard\index.html

Runner script: e:\signal_agent\app\agents\social_offload\social_offload.py

Logs JSONL: e:\signal_agent\data\social_offload\logs\social_offload_runs.jsonl

Outputs dir: e:\signal_agent\data\social_offload\outputs\linkedin\

Passing fixture: e:\signal_agent\tests\fixtures\social_offload\core_artifact.md

Failing fixture: e:\signal_agent\tests\fixtures\social_offload\core_artifact_failing.md


GOAL:

Make the dashboard verification checklist mechanically verifiable (no reliance on UI refresh/count). Provide deterministic commands to validate:
```

1) log append correctness

2) output directory timestamp delta correctness

3) REPO_ROOT correctness (preflight)

Also specify what to paste back (stdout_tail/stderr_tail + sanity-check output) if something fails.

REQUIREMENTS:

A) Preflight Sanity Check (REPO_ROOT correctness)

Provide a PowerShell command that prints:

- DASHBOARD_DIR

- REPO_ROOT (must equal e:\signal_agent)

- SOCIAL_OFFLOAD script path (must equal e:\signal_agent\app\agents\social_offload\social_offload.py)

Use exactly this logic:

```
python -c "from pathlib import Path; p=Path(r'app\hq\dashboard\hq_dashboard.py').resolve(); d=p.parent; print('DASHBOARD_DIR=',d); print('REPO_ROOT=',d.parents[3]); print('SOCIAL_OFFLOAD=',d.parents[3]/'app'/'agents'/'social_offload'/'social_offload.py')"
```

B) Mechanically verifiable log append checks

After each dashboard run, verify the newest JSONL entry:

- artifact_path matches the fixture path used

- status is ok vs fault correctly

- timestamp is recent (within last 5 minutes)

Provide PowerShell commands to:

- read the last line of social_offload_runs.jsonl

- parse it as JSON

- print key fields (timestamp, artifact_path, status, artifact_id, render_id, fault_codes if present)

- compare artifact_path and status to expected values and print PASS/FAIL

C) Mechanically verifiable outputs delta checks

Before running the failing fixture, record the newest output file creation/modification time under:

e:\signal_agent\data\social_offload\outputs\linkedin\

After running failing fixture, re-check that the newest output timestamp did NOT advance.

Print PASS/FAIL.

D) Updated Verification Checklist

Output an updated checklist that includes:

1) Start dashboard

2) Preflight sanity check

3) Run passing fixture via UI → validate returncode + log append + output created (timestamp advanced)

4) Run failing fixture via UI → validate nonzero returncode + log append (fault) + output timestamp unchanged

E) Failure Triage Instructions

If any step fails, instruct me to paste back:

- dashboard stdout_tail

- dashboard stderr_tail

- output of the REPO_ROOT sanity command

- last JSONL line printed by the log check commands

```
OUTPUT FORMAT:

Return ONLY:

- A single updated checklist (numbered steps)

- A single block of PowerShell commands that implement A/B/C checks
(clearly labeled)

No additional commentary.
```

ok thats going in codex high. what should i look for? and how can i proceede in antigravity? also adding codex pics. i want to make sure to take advantage of what i have access to.

app/app/

```
Audit app/hq/dashboard/hq_dashboard.py for:

1) incorrect repo-root derivation

2) subprocess working-directory errors

3) absolute path mishandling

4) missing error propagation

Return a patch diff only.
```

```
Generate a deterministic CLI test script that:

1) runs passing fixture

2) runs failing fixture

3) asserts JSONL appended correctly

4) asserts output file created only for pass

Return a single Python script.
```

```
Create a regression test module under tests/integration/ that
programmatically:

- launches social_offload.py

- verifies JSONL append

- verifies output delta

- verifies failing artifact produces no output file

Use tempfile snapshots if needed.

Return a complete runnable pytest file.
```

Help you design the multi-channel abstraction cleanly

give you the exact Codex high prompt to run right now along with a antigravity prompt

help you design the Phase 2 dashboard architecture

```
ChannelSpec
```

```
channel_id "linkedin" | "facebook" | "x" | "substack" | "youtube" | "reddit"
pack_path constraints/packs/domain/<channel>_pack.yaml
```

```
template_dir app/agents/social_offload/templates/<channel>/
template_file     post.jinja
output_ext .txt  .md
output_root data/social_offload/outputs/<channel>/
boundary_conditions
```

```
social_offload.py

--channel <id>
      ChannelSpec
```

```
app/agents/social_offload/channel_registry.py

          { "linkedin": ChannelSpec(...), ... }
```

```
artifact_id = sha256(content)[:12]
render_id =
sha256(f"{artifact_id}_{pack_version}_{template_version}_{channel}")
[:12]
channel    render_id
```

```
social_offload_runs.jsonl

channel
template_name
template_version
pack_path
```

`out_file`

📦

```
You are operating inside repo: E:\signal_agent


Task: Refactor social_offload into a clean multi-channel abstraction while
preserving existing behavior for LinkedIn.


Hard constraints:

- Do NOT change constraint evaluation semantics (reproject_checkpoint
fail-closed).

- Do NOT change output determinism for LinkedIn except adding channel into
render_id input to avoid cross-channel collisions.

- Preserve existing LinkedIn paths and fixtures, and ensure all existing
tests still pass or are updated with equivalent golden snapshots.

- No speculative features. Make smallest viable change set.


Required deliverables:

1) Create app/agents/social_offload/channel_registry.py with a
deterministic registry:

  - ChannelSpec dataclass
```

- registry mapping for "linkedin" (and stubs for "facebook","x","substack","youtube","reddit" with TODO pack/template paths but not required to work yet)

- Each spec includes: channel_id, pack_path, template_dir, template_file, output_root, output_ext, pack_version, template_version

2) Update app/agents/social_offload/social_offload.py to:

- accept --channel choices from registry keys

- load channel spec

- use spec.pack_path if --pack not provided (but allow override via --pack to keep current CLI working)

- select template via spec.template_dir/spec.template_file

- write to spec.output_root/channel_id/artifact_id/render_id.ext (or keep existing linkedin output tree but under outputs/linkedin/<artifact_id>/<render_id>.txt)

- log JSONL includes channel, pack_version, template_version, template_file, out_file on success

3) Add regression tests under tests/integration/test_social_offload_multichannel.py:

- run passing fixture for linkedin and assert:

  a) return code 0

  b) JSONL last entry status ok and channel linkedin

  c) output file exists

- run failing fixture for linkedin and assert:

  a) nonzero return code

  b) JSONL last entry status fault and channel linkedin

  c) no new output file created by timestamp delta

Use pathlib + subprocess, no external deps beyond pytest.

```
4) Patch HQ dashboard backend if needed to align with any moved paths (but
keep endpoints stable).


Output format:

- Provide a patch/diff only (unified diff).

- No commentary.
```

📦

```
TASK: Generate Phase-2 social_offload channel packs + templates (bounded,
deterministic)


Context:

Existing reference pack: constraints/packs/domain/linkedin_pack.yaml

Existing reference template:
app/agents/social_offload/templates/linkedin_post.jinja

Existing runner writes outputs under:
data/social_offload/outputs/<channel>/<artifact_id>/<render_id>.<ext>


Goal:

Create domain packs + templates for:

- facebook

- x

- substack

- youtube

- reddit
```

Constraints:

- Do not change LinkedIn artifacts.
- Each channel pack must define boundary_conditions appropriate to the channel:
  - facebook: max_word_count 120, max_claims 3
  - x: max_word_count 60, max_claims 1, disallow hashtags > 2
  - substack: max_word_count 600, max_claims 6
  - youtube: max_word_count 900, max_claims 6 (script tone), include section headers allowed
  - reddit: max_word_count 350, max_claims 5, disallow “link in bio” type phrases
- Each pack includes disallowed_phrases tailored to the channel (avoid emojis for linkedin, more permissive for facebook, etc.)
- All packs remain fail-closed via your constraint engine (DENY when violations).
- Templates must render from the same extracted state format (sections + claims).
- Output extension:
  - facebook: .txt
  - x: .txt
  - substack: .md
  - youtube: .txt
  - reddit: .md

Deliverables:

- constraints/packs/domain/facebook_pack.yaml
- constraints/packs/domain/x_pack.yaml
- constraints/packs/domain/substack_pack.yaml
- constraints/packs/domain/youtube_pack.yaml

```
- constraints/packs/domain/reddit_pack.yaml

- templates folder structure under
app/agents/social_offload/templates/<channel>/

 with one primary Jinja template per channel:

   post.jinja (or script.jinja for youtube)


Output format:

- File tree

- Full contents of each file

- No commentary outside technical output
```

`data/social_offload/staging.jsonl` `staging.sqlite`

```
GET /api/channels
GET /api/rendered?channel=&artifact_id=&render_id=

POST /api/stage
GET /api/stage
```

this i what cli said newest_mtime: float | None

```
def _snapshot_outputs(linkedin_dir: Path) -> OutputSnapshot:
    txt_files = sorted((p for p in linkedin_dir.rglob("*.txt") if p.is_file()), key=lambda p:
p.as_posix())
    newest_mtime = max((p.stat().st_mtime for p in txt_files), default=None)
    return OutputSnapshot(files=tuple(str(p.resolve()) for p in txt_files),
newest_mtime=newest_mtime)


def _read_jsonl_entries(jsonl_path: Path) -> list[dict[str, Any]]:
    if not jsonl_path.exists():
        return []

    entries: list[dict[str, Any]] = []
    for raw in jsonl_path.read_text(encoding="utf-8").splitlines():
        line = raw.strip()
        if not line:
            continue
        entries.append(json.loads(line))
    return entries


def _run_social_offload(artifact_path: Path, out_dir: Path) ->
subprocess.CompletedProcess[str]:
    cmd = [
        sys.executable,
        str(SOCIAL_OFFLOAD_SCRIPT),
        "--artifact",
        str(artifact_path.resolve()),
        "--channel",
```

```
        "linkedin",
        "--out",
        str(out_dir),
        "--pack",
        str(PACK_PATH.resolve()),
    ]
    return subprocess.run(
        cmd,
        cwd=str(REPO_ROOT),
        text=True,
        capture_output=True,
        check=False,
    )


@pytest.fixture(autouse=True)
def _require_inputs_exist() -> None:
    for path in (SOCIAL_OFFLOAD_SCRIPT, PACK_PATH, PASSING_FIXTURE,
FAILING_FIXTURE):
        assert path.exists(), f"Required path missing: {path}"


def test_social_offload_passing_appends_ok_log_and_advances_output(tmp_path:
Path) -> None:
    out_dir = tmp_path / "outputs"
    logs_jsonl = tmp_path / "logs" / "social_offload_runs.jsonl"
    linkedin_dir = out_dir / "linkedin"

    before_entries = _read_jsonl_entries(logs_jsonl)
    before_snapshot = _snapshot_outputs(linkedin_dir)

    result = _run_social_offload(PASSING_FIXTURE, out_dir)
    assert result.returncode == 0, f"stdout={result.stdout}\nstderr={result.stderr}"

    after_entries = _read_jsonl_entries(logs_jsonl)
    assert len(after_entries) == len(before_entries) + 1

    last = after_entries[-1]
    assert last.get("artifact_path") == str(PASSING_FIXTURE.resolve())
    assert last.get("status") == "ok"
```

```
    assert last.get("artifact_id")
    assert last.get("render_id")
    assert isinstance(last.get("timestamp"), (int, float))

    after_snapshot = _snapshot_outputs(linkedin_dir)
    assert len(after_snapshot.files) > len(before_snapshot.files)
    assert after_snapshot.newest_mtime is not None
    if before_snapshot.newest_mtime is not None:
        assert after_snapshot.newest_mtime > before_snapshot.newest_mtime


def test_social_offload_failing_appends_fault_log_and_creates_no_output(tmp_path:
Path) -> None:
    out_dir = tmp_path / "outputs"
    logs_jsonl = tmp_path / "logs" / "social_offload_runs.jsonl"
    linkedin_dir = out_dir / "linkedin"

    before_entries = _read_jsonl_entries(logs_jsonl)
    before_snapshot = _snapshot_outputs(linkedin_dir)

    result = _run_social_offload(FAILING_FIXTURE, out_dir)
    assert result.returncode != 0, f"expected non-zero returncode,
stdout={result.stdout}\nstderr={result.stderr}"

    after_entries = _read_jsonl_entries(logs_jsonl)
    assert len(after_entries) == len(before_entries) + 1

    last = after_entries[-1]
    assert last.get("artifact_path") == str(FAILING_FIXTURE.resolve())
    assert last.get("status") == "fault"
    assert last.get("artifact_id")
    assert last.get("render_id")
    assert last.get("fault_codes")
    assert isinstance(last.get("timestamp"), (int, float))

    after_snapshot = _snapshot_outputs(linkedin_dir)
    assert after_snapshot.files == before_snapshot.files
    assert after_snapshot.newest_mtime == before_snapshot.newest_mtime
```

Could not execute this test in the current shell because pytest is not installed in the active Python environment (No module named pytest).

`pytest`

```
OpenAI Codex (v0.104.0)
model: gpt-5.3-codex
directory: ~
```

```
!python -c "import sys; print(sys.executable)"
```

```
!python -m pip install pytest
```

`pip install pytest`

```
python -m pip install ...
```

```
!python -m pytest --version
```

```
E:\signal_agent
```

```
cd E:\signal_agent
python -m venv .venv
.\.venv\Scripts\activate
python -m pip install --upgrade pip
python -m pip install pytest
```

```
!python -m pytest -q
```

```
C:\Users\mrcol\AppData\Local\Python\pythoncore-3.12-64\python.exe
```

```
cd E:\signal_agent
python -m venv .venv
.\.venv\Scripts\activate
python -m pip install --upgrade pip
python -m pip install pytest
```

```
python -c "import sys; print(sys.executable)"
```

```
E:\signal_agent\.venv\Scripts\python.exe
```

```
python -m pip install pytest
```

```
C:\Users\mrcol\AppData\Local\Python\pythoncore-3.12-64\
```

```
python -m pytest -q
```

```
python -c "import sys; print(sys.executable)"
```

```
PS C:\Users\mrcol>
```

```
C:\Users\mrcol\AppData\Local\Python\pythoncore-3.12-64\python.exe
```

```
(.venv) PS E:\signal_agent>
```

(.venv)

```
cd E:\signal_agent

.\.venv\Scripts\activate
```

```
python -c "import sys; print(sys.executable)"
```

```
E:\signal_agent\.venv\Scripts\python.exe
```

```
python -m pip install pytest
```

```
python -m pytest --version
```

```
python -m pytest -q
```

`requirements-dev.txt`

```
pytest
```

```
python -m pip install -r requirements-dev.txt
```

⚠

is that right

```
(.venv) PS E:\signal_agent> ✅
        python -c "import sys; print(sys.executable)" ✅
```

```
E:\signal_agent\.venv\Scripts\python.exe
```

```
python -m pip install pytest
python -m pytest --version
python -m pytest -q
```

`sys.executable` `.venv`

⚠

`(.venv)`

`ModuleNotFoundError: No module named 'yaml'`

`yaml`

`PyYAML`

📦 `(.venv) PS E:\signal_agent>`

```
python -m pip install PyYAML
python -c "import yaml; print('yaml ok:', yaml.__version__)"
python -m pytest -q
```

`pythoncore-3.12-64`

`(.venv)` `PS E:\signal_agent>`

`jinja2`
`pydantic` `fastapi` `uvicorn`

```
python -m pip install <PACKAGE_NAME>
```

```
python -m pip freeze > requirements-dev.lock.txt
```

`requirements.txt` `requirements-dev.txt`

Thought for 50s

`social_offload.py`

`python -m pip install <PACKAGE_NAME>`

`<>`

`<PLACEHOLDER>`

📦

```
# Use a real package name, no angle brackets
python -m pip install PyYAML
```

`after_entries == before_entries + 1`

`argparse` `--out`

`2`

[]

📦

```
cd E:\signal_agent
.\.venv\Scripts\activate

python -m pytest -vv -s
tests\integration\test_social_offload_regression.py::test_social_offload_failing_appends_fault_log_and_creates_no_output
```

```
error: unrecognized arguments: --out ...
  usage: social_offload.py ...
```

--out

📦

```
cd E:\signal_agent
.\.venv\Scripts\activate
```

```
$OUT = "E:\signal_agent\.tmp_offload_out"
Remove-Item -Recurse -Force $OUT -ErrorAction SilentlyContinue
New-Item -ItemType Directory -Force $OUT | Out-Null

python E:\signal_agent\app\agents\social_offload\social_offload.py `
 --artifact E:\signal_agent\tests\fixtures\social_offload\core_artifact.md
`
 --channel linkedin `
 --pack E:\signal_agent\constraints\packs\domain\linkedin_pack.yaml `
 --out $OUT

Write-Host "=== logs ===" 
Get-ChildItem -Recurse ($OUT + "\..\logs") -ErrorAction SilentlyContinue

Write-Host "=== outputs ===" 
Get-ChildItem -Recurse $OUT -ErrorAction SilentlyContinue
```

`--out`

`--out` `social_offload.py`

`--out`

`argparse` `--out`

`run_social_offload(... out_dir ...)`

`out_dir.parent / "logs"` `out_dir / "../logs"`

`tmp_path/logs/social_offload_runs.jsonl`

`E:\signal_agent\data\social_offload\logs\social_offload_runs.jsonl`

`--out`

`pythoncore-3.12-64`

`(.venv)`

⚠️

⚠️

`social_offload.py` `ModuleNotFoundError: No module named 'jinja2'`

`stderr`

`.venv`

`jinja2` `yaml` `PyYAML`

📦

```
cd E:\signal_agent
.\.venv\Scripts\activate

python -m pip install jinja2
```

```
python -m pip install -r E:\signal_agent\app\hq\dashboard\requirements.txt
```

jinja2 social_offload.py fastapi uvicorn

📦

```
cd E:\signal_agent
.\.venv\Scripts\activate

$OUT = "E:\signal_agent\.tmp_offload_out"
Remove-Item -Recurse -Force $OUT -ErrorAction SilentlyContinue
New-Item -ItemType Directory -Force $OUT | Out-Null

python E:\signal_agent\app\agents\social_offload\social_offload.py `
 --artifact E:\signal_agent\tests\fixtures\social_offload\core_artifact.md
`
 --channel linkedin `
 --pack E:\signal_agent\constraints\packs\domain\linkedin_pack.yaml `
 --out $OUT
```

`tmp_path / "logs" / "social_offload_runs.jsonl"`

`social_offload.py`

`out_dir.parent / "logs" / "social_offload_runs.jsonl"`

`out_dir = tmp_path / "outputs"`

`tmp_path / "logs" / "social_offload_runs.jsonl"` ✅

`out_dir.parent`

`E:\signal_agent\data\social_offload\logs\...`

`social_offload.py`

`--out` `--out`

`--out` `out_dir.parent`

`E:\signal_agent\app\agents\social_offload\social_offload.py`

```
parser.add_argument("--out", ...)
log_dir = out_dir.parent / "logs"
```

📦

```
python -c "import re;
p=r'E:\signal_agent\app\agents\social_offload\social_offload.py';
s=open(p,'r',encoding='utf-8').read(); print('HAS --out:', '--out' in s);
print('USES out_dir.parent/logs:',
bool(re.search(r'out_dir\.parent\s*/\s*[\"'']logs[\"'']', s)));"
```

📦

```
cd E:\signal_agent
.\.venv\Scripts\activate

python -m pytest -vv -s
tests\integration\test_social_offload_regression.py::test_social_offload_failing_appends_fault_log_and_creates_no_output
```

`_run_social_offload()`

`-s`

📦

```
def _run_social_offload(artifact_path: Path, out_dir: Path) ->
subprocess.CompletedProcess[str]:
    cmd = [
        sys.executable,
        str(SOCIAL_OFFLOAD_SCRIPT),
        "--artifact",
        str(artifact_path.resolve()),
        "--channel",
        "linkedin",
        "--out",
        str(out_dir),
        "--pack",
```

```
        str(PACK_PATH.resolve()),
    ]
    result = subprocess.run(
        cmd,
        cwd=str(REPO_ROOT),
        text=True,
        capture_output=True,
        check=False,
    )
    # Always print when running with -s so failures show root cause
    if result.stdout:
        print("\n--- social_offload stdout ---\n", result.stdout)
    if result.stderr:
        print("\n--- social_offload stderr ---\n", result.stderr)
    return result
```

`jinja2`

`--out`

⚠

`$OUT`

📦

```
cd E:\signal_agent
.\.venv\Scripts\activate


$OUT = "E:\signal_agent\.tmp_offload_out"
Remove-Item -Recurse -Force $OUT -ErrorAction SilentlyContinue
New-Item -ItemType Directory -Force $OUT | Out-Null
```

```
python E:\signal_agent\app\agents\social_offload\social_offload.py `
 --artifact
E:\signal_agent\tests\fixtures\social_offload\core_artifact_failing.md `
 --channel linkedin `
 --pack E:\signal_agent\constraints\packs\domain\linkedin_pack.yaml `
 --out $OUT

echo "Exit code: $LASTEXITCODE"
```

```
out_dir.parent / "logs"
```

```
$OUT = E:\signal_agent\.tmp_offload_out
```

```
E:\signal_agent\logs\social_offload_runs.jsonl
```

📦

```
Get-ChildItem E:\signal_agent\logs
Get-Content E:\signal_agent\logs\social_offload_runs.jsonl
```

```
"status": "fault"
"artifact_path"
```

`"fault_codes"`

`$OUT\linkedin\...`

```
after_entries == before_entries + 1
```

📦

```
python -m pytest -vv -s
tests\integration\test_social_offload_regression.py::test_social_offload_f
ailing_appends_fault_log_and_creates_no_output
```

`social_offload_runs.jsonl`

⚠️

⚠️

✅

❌

❌ `ModuleNotFoundError`

❌

❌

`social_offload.py`

```
[INFO] Starting linkedin social offload ...

[INFO] Artifact ID ...
```

```
Pipeline completed successfully.
```

```
assert len(after_entries) == len(before_entries) + 1

0 == (0 + 1)
```

```
echo "Exit code: $LASTEXITCODE"
```

`social_offload.py`

```
violations = validate_against_pack(...)
if violations:
   logger.error(...)
   return
```

```
sys.exit(1)
```

`$LASTEXITCODE`

⚠

`Exit code: 1` ✅
✅
❌

`E:\signal_agent\logs\social_offload_runs.jsonl`
`Get-ChildItem` `.jsonl` `Get-Content` `.jsonl`

`artifact_path`
`status: "fault"`
`fault_codes`

`out_dir.parent / "logs"`

`data/social_offload/logs/social_offload_runs.jsonl`

`logs/social_offload_runs.jsonl`

`data/social_offload/logs/social_offload_runs.jsonl`
`E:\signal_agent\logs\social_offload_runs.jsonl`

`.jsonl`
`.jsonl` `.json`

`--out`
`tmp_path/logs/...`
`E:\signal_agent\logs/...`

`data/social_offload/logs/social_offload_runs.jsonl`
`--log`

```
--out
--out <dir>
<dir>/linkedin/...
<dir>/logs/social_offload_runs.jsonl
--out data/social_offload/outputs
```

```
You are operating inside repo root: E:\signal_agent.


GOAL:

Unify the social_offload logging contract so:

1) The CLI, dashboard, and tests all read/write the same JSONL log file.

2) Failing runs append a "fault" record and exit non-zero (already true).

3) Passing runs append an "ok" record and write output files.

4) The dashboard "Recent Runs" and "Generated Output Files" reflect truth
without UI caching ambiguity.


CURRENT STATE (observed):

- social_offload failing run prints "Failing closed due to constraint
violation ..." and returns exit code 1.

- social_offload writes JSONL entries to
E:\signal_agent\logs\social_offload_runs.jsonl (verified by Get-Content).
```

- HQ dashboard backend reads logs from: data/social_offload/logs/social_offload_runs.jsonl (in app/hq/dashboard/hq_dashboard.py).

- Dashboard outputs dir is: data/social_offload/outputs/linkedin (in app/hq/dashboard/hq_dashboard.py).

- Integration test generated earlier assumed logs live under tmp_path/logs/social_offload_runs.jsonl and outputs under tmp_path/outputs/linkedin.

TASKS:

A) Search for where the social_offload pipeline sets LOG paths.

  - Find in: app/agents/social_offload/social_offload.py and any shared logging util.

  - Determine if there is a hardcoded logs path (e.g., REPO_ROOT/logs) or env-driven.

B) Choose ONE canonical contract (prefer Option A):

  - Canonical log path: E:\signal_agent\data\social_offload\logs\social_offload_runs.jsonl

  - Canonical outputs root: E:\signal_agent\data\social_offload\outputs

  - Channel output: <outputs_root>\linkedin\<artifact_id>\<render_id>.txt

C) Implement:

  1) Add CLI args:

     --out <dir> (already exists) controls outputs root.

     --log <path> (new) controls JSONL log path.

     If --log not provided, default to canonical log path above.

     Ensure parent dirs are created.

  2) Ensure BOTH ok and fault paths append JSONL entry with:

```
     timestamp, artifact_path, artifact_id, render_id, channel, status,
fault_codes (if fault), word_count/claim_count (if ok),
pack_version/template_version (if available).

  3) Ensure exit code remains:

     ok -> 0

     fault -> 1


D) Update HQ dashboard backend (app/hq/dashboard/hq_dashboard.py):

  - When calling social_offload subprocess, pass:

    --out E:\signal_agent\data\social_offload\outputs

    --log
E:\signal_agent\data\social_offload\logs\social_offload_runs.jsonl

  - Keep /api/runs reading that same file.

  - Keep /api/outputs listing from outputs/linkedin.


E) Update/author regression tests so they reflect the real contract.

  - If tests should be hermetic, they should pass --out tmp_path/outputs
and --log tmp_path/logs/social_offload_runs.jsonl.

  - The test should then read that tmp_path log, not repo-global logs.


F) Provide a short verification procedure:

  - CLI passing fixture run + CLI failing fixture run.

  - Confirm newest JSONL line matches run and outputs are or are not
written.

  - Confirm dashboard shows both runs.


OUTPUT:

- Make the code changes directly.

- List touched files with diffs.
```

```
- Include a one-shot PowerShell verification script (copy/paste ready).
```

```
ANTIGRAVITY // TASK: SocialOffload Log Contract Unification (v1)


State□:

- social_offload CLI writes logs to repo-root
\logs\social_offload_runs.jsonl

- HQ dashboard reads logs from
\data\social_offload\logs\social_offload_runs.jsonl

- Tests assumed logs co-locate under --out/tmp_path

- Behavior is correct (fault exits 1 and appends), but routing is
incoherent across layers.


Transformation:

1) Define canonical log contract:

  - outputs_root := REPO_ROOT\data\social_offload\outputs

  - log_path :=
REPO_ROOT\data\social_offload\logs\social_offload_runs.jsonl

2) Add CLI arg --log with default = canonical log_path.

3) Ensure ok/fault both append JSONL line to log_path with required
fields.

4) Ensure dashboard subprocess passes explicit --out + --log to guarantee
alignment.

5) Ensure tests pass explicit --out + --log pointing to tmp_path for
hermetic verification.

6) Add mechanical verification script:

  - capture log tail before/after

  - capture outputs newest mtime before/after
```

```
  - assert deltas by timestamp not count


State□+1:

- One log file, one JSONL format, one source of truth

- Dashboard UI reflects exact log/output truth

- Regression tests are mechanically verifying the same contract

- Fail-closed semantics are enforced and observable (exit code + log
append)
```

`.venv` `python -c "import sys; print(sys.executable)"`

`yaml` `jinja2`

`social_offload.py`

`--out`

“I don't really know what you mean when you say if we want a paste pass shown to your social off, oh, never mind. So you just want me to go on to socialoffload.py and then give you the top 60 lines?”

00:12

“So I'm good to do the CLI codex prompt and then the anti-gravity prompt, and then go from there. I'll just give you the screenshots like I have been.”

00:11

“Cool, this is awesome. So I come sitting in the command chair.”

00:05

proceede?

`sys.path.insert(...)`

`sys.path`

```
1
```

I have successfully unified the Social Offload Log Contract across the CLI, HQ Dashboard, and tests following a deterministic single source of truth format.

Key Changes:

Added the --log argument to social_offload.py allowing default output to the absolute path ROOT/data/social_offload/logs/social_offload_runs.jsonl.
Updated the FastAPI hq_dashboard.py to pass the explicit --out and --log mappings into the subprocess to ensure exact parity with the new log contract.
Rewrote the _run_social_offload subprocess handler inside test_social_offload_regression.py to route its outputs and JSONL streams hermetically to a tmp_path fixture space, avoiding test pollution of the main operational logs.
Created and executed tests/integration/verify_social_offload_contract.py, a robust mechanical tool verifying both the passing and failing fixtures by directly observing timestamps within the JSON tail stream and the OS mtime file modifications.
The contract is solid. When constraints pass, the artifact is written and a JSONL record append hits with an ok state. When constraints fail (e.g. character limits or forbidden phrases), the system fails closed (exit code 1), writes no generation file to disk, and pushes the associated fault codes into the JSONL trace log simultaneously. this is what anti grav said

1. Single canonical log path
2. `ROOT/data/social_offload/logs/social_offload_runs.jsonl`

`--out`
`--log`

`--out` `--log`

`tmp_path` `--out`
`tmp_path/logs/...jsonl` `--log`

`{status: "ok"}`
`{status: "fault"}`

`status == "ok"`

`status == "fault"`

```
python -m pytest -vv
```

`--log`

`--log`

Good.

That is exactly what you wanted structurally.

Let's strip the emotion out and check the mechanics.

You now have:

1. **Single canonical log path**

```
ROOT/data/social_offload/logs/social_offload_runs.jsonl
```

2. CLI:

   * `--out` controls output root
   * `--log` controls JSONL destination
   * Defaults to canonical path

3. Dashboard:

   * Explicitly passes both `--out` and `--log`
   * Reads the same JSONL file

4. Tests:

   * Pass hermetic `tmp_path` for `--out`
   * Pass hermetic `tmp_path/logs/...jsonl` for `--log`
   * No cross-contamination of operational logs

5. Verified contract:

   * Passing → exit 0 → output written → JSONL `{status: "ok"}`
   * Failing → exit 1 → no output → JSONL `{status: "fault"}`

That is a clean fail-closed governance contract.

Now we do not assume it is solid.

We verify invariants.

Run these three checks:

1) CLI invariant

Manual passing run → tail JSON:

* New line appended
* `status == "ok"`
* Output directory contains new render

Manual failing run:

* Exit code 1
* No new output directory created
* JSON appended with `status == "fault"`

You already saw this, but confirm once more post-refactor.

---

2) Dashboard invariant

From UI:

* Run passing artifact
* Run failing artifact
* Refresh

Check:

* Recent Runs shows both
* Generated Files only shows passing
* No duplicate ghost entries

---

3) Test invariant

Run full integration suite:

```
python -m pytest -vv
```

Ensure:

* No failures
* No path leakage into production logs

If all three pass, then yes:

You have a unified deterministic log contract.

And that matters more than the feature itself.

You now have:

Validation → Structured Log → Exit Code → Artifact Side Effect

Bounded.
Observable.
Auditable.

That's real system coherence.

Small note:
Now that you've introduced `--log`, consider one additional invariant for the future:

If `--log` parent directory does not exist → create it.
No implicit assumptions.

You're not just "in the command chair."

You've reduced a three-headed logging ambiguity into one canonical stream.

That's system maturity.

Now the next question isn't "does it work?"

It's:

What invariant do you want to harden next? can you give me more instructions for this?

`json.dumps(obj) + "\n"`

`corrupt_line_count`

`artifact_id` `render_id` `run_id`

`(timestamp_ns, pid, artifact_id)`

`run_id` `social_offload.py`

`run_id`

`.txt`

`.../render_id.txt.tmp`
`fsync` `render_id.txt`
`status=ok`

`.txt`
`.tmp`

`fault`

`.tmp`
`fault_codes`

`contract_version`

`/api/runs`

`file_size_bytes`
`last_mtime`
`corrupt_line_count`
`last_run_id`

`ROOT/data/...`

```
cd E:\signal_agent

.\.venv\Scripts\activate
```

```
# 1) Full suite
python -m pytest -vv

# 2) Check canonical log hasn't been touched by hermetic tests (run AFTER pytest)
Get-Item .\data\social_offload\logs\social_offload_runs.jsonl | Select-Object FullName, Length, LastWriteTime

# 3) Run two CLI executions in parallel to stress JSONL append (adjust --log to canonical)
$LOG="E:\signal_agent\data\social_offload\logs\social_offload_runs.jsonl"
$OUT="E:\signal_agent\.tmp_offload_out"
Remove-Item -Recurse -Force $OUT -ErrorAction SilentlyContinue
New-Item -ItemType Directory -Force $OUT | Out-Null

Start-Job { python E:\signal_agent\app\agents\social_offload\social_offload.py --artifact E:\signal_agent\tests\fixtures\social_offload\core_artifact.md --channel linkedin --pack E:\signal_agent\constraints\packs\domain\linkedin_pack.yaml --out E:\signal_agent\.tmp_offload_out --log E:\signal_agent\data\social_offload\logs\social_offload_runs.jsonl } | Out-Null
Start-Job { python E:\signal_agent\app\agents\social_offload\social_offload.py --artifact E:\signal_agent\tests\fixtures\social_offload\core_artifact_failing.md --channel linkedin --pack E:\signal_agent\constraints\packs\domain\linkedin_pack.yaml --out E:\signal_agent\.tmp_offload_out --log E:\signal_agent\data\social_offload\logs\social_offload_runs.jsonl } | Out-Null

Get-Job | Wait-Job | Receive-Job
```

```
Get-Job | Remove-Job


# 4) Validate JSONL parseability (fails fast if any line is corrupt JSON)
python -c "import json;
p=r'E:\signal_agent\data\social_offload\logs\social_offload_runs.jsonl';
bad=0; n=0;
from pathlib import Path;
for line in Path(p).read_text(encoding='utf-8').splitlines():
   if not line.strip():
       continue
   n+=1
   try: json.loads(line)
   except Exception: bad+=1
print('lines=',n,'bad=',bad);
raise SystemExit(1 if bad else 0)"
```

---

1 or 2?

```
1
```

1

/api/runs
/api/outputs

```
PASS_RC=0

FAIL_RC=1

...

DASHBOARD_RUNS_HAS_PASS=PASS

DASHBOARD_RUNS_HAS_FAIL=PASS

DASHBOARD_OUTPUTS_HAS_PASS_FILE=PASS
```

```
run_id = uuid4()
```

`$checks`

```
PASS_RC_IS_ZERO

FAIL_RC_IS_NONZERO
```

```
PASS_LOG_ARTIFACT_MATCH

FAIL_LOG_ARTIFACT_MATCH
```

```
PASS_LOG_STATUS_OK

FAIL_LOG_STATUS_FAULT
```

```
OUTPUT_ADVANCED_AFTER_PASS

NO_NEW_OUTPUT_AFTER_FAIL
```

```
DASHBOARD_RUNS_HAS_PASS

DASHBOARD_RUNS_HAS_FAIL

DASHBOARD_OUTPUTS_HAS_PASS_FILE
```

🔐

```
write temp file
fsync
rename to final
append JSONL
```

```
flush JSONL
```

🧵

🆔

```
run_id = uuid4()
```

A) Harden atomicity + concurrency

`os.replace(tmp, final)`

`"ok"`

`"fault"`

`*.tmp`

`E:\signal_agent\app\utils\io_contract.py`

```
# app/utils/io_contract.py
from __future__ import annotations


import json
import os
import random
import time
import uuid
from dataclasses import dataclass
from pathlib import Path
```

```
from typing import Any, Iterable, Optional


IS_WINDOWS = os.name == "nt"


if IS_WINDOWS:
    import msvcrt  # type: ignore
else:
    import fcntl  # type: ignore



@dataclass(frozen=True)
class AtomicWriteResult:
    final_path: Path
    tmp_path: Path



def ensure_parent_dir(path: Path) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)



def atomic_write_text(final_path: Path, text: str, encoding: str =
"utf-8") -> AtomicWriteResult:
    """
    Writes to final_path via temp file + fsync + atomic replace.
    Guarantees final_path is either old or fully new, never partial.
    """
    ensure_parent_dir(final_path)
    tmp_path = final_path.with_name(final_path.name +
f".tmp.{uuid.uuid4().hex}")
```

```
    # Write temp
    with open(tmp_path, "w", encoding=encoding, newline="\n") as f:
        f.write(text)
        f.flush()
        os.fsync(f.fileno())

    # Atomic commit
    os.replace(tmp_path, final_path)
    return AtomicWriteResult(final_path=final_path, tmp_path=tmp_path)



class FileLock:
    """
    Cross-platform advisory lock using a dedicated lockfile.
    - Windows: msvcrt.locking
    - POSIX: fcntl.flock
    """
    def __init__(self, lock_path: Path) -> None:
        self.lock_path = lock_path
        self._fh = None

    def __enter__(self) -> "FileLock":
        ensure_parent_dir(self.lock_path)
        self._fh = open(self.lock_path, "a+b")
        if IS_WINDOWS:
            # Lock 1 byte from start; hold while open.
            self._fh.seek(0)
```

```
            msvcrt.locking(self._fh.fileno(), msvcrt.LK_LOCK, 1)
        else:
            fcntl.flock(self._fh.fileno(), fcntl.LOCK_EX)
        return self

    def __exit__(self, exc_type, exc, tb) -> None:
        assert self._fh is not None
        try:
            if IS_WINDOWS:
                self._fh.seek(0)
                msvcrt.locking(self._fh.fileno(), msvcrt.LK_UNLCK, 1)
            else:
                fcntl.flock(self._fh.fileno(), fcntl.LOCK_UN)
        finally:
            self._fh.close()
            self._fh = None



def append_jsonl_atomic(
    jsonl_path: Path,
    record: dict[str, Any],
    *,
    lock_path: Optional[Path] = None,
    retries: int = 30,
    base_sleep_s: float = 0.02,
) -> None:
    """
    Concurrent-safe JSONL append:
```

```
    - locks a lockfile
    - appends one line
    - flush + fsync
  """
  ensure_parent_dir(jsonl_path)
  if lock_path is None:
      lock_path = jsonl_path.with_suffix(jsonl_path.suffix + ".lock")

  payload = json.dumps(record, ensure_ascii=False) + "\n"

  # Simple retry loop for lock contention (esp. Windows + antivirus +
slow IO)
  last_err: Optional[BaseException] = None
  for i in range(retries):
      try:
          with FileLock(lock_path):
              with open(jsonl_path, "a", encoding="utf-8", newline="\n")
as f:
                  f.write(payload)
                  f.flush()
                  os.fsync(f.fileno())
          return
      except BaseException as e:
          last_err = e
          # Jittered backoff
          sleep = base_sleep_s * (1.2 ** i) + random.random() *
base_sleep_s
          time.sleep(min(sleep, 0.5))
```

```
    raise RuntimeError(f"append_jsonl_atomic failed after {retries} retries: {last_err}") from last_err
```

---

social_offload.py

social_offload.py

atomic_write_text()
append_jsonl_atomic()
"ok"
--log
run_id

"ok"

"fault"

---

tests/integration/test_social_offload_concurrency.py

--log

```
# tests/integration/test_social_offload_concurrency.py
from __future__ import annotations


import json
import os
import subprocess
```

```
import sys

from pathlib import Path


REPO_ROOT = Path(__file__).resolve().parents[2]

SCRIPT = REPO_ROOT / "app" / "agents" / "social_offload" /
"social_offload.py"

PACK = REPO_ROOT / "constraints" / "packs" / "domain" /
"linkedin_pack.yaml"

PASSING = REPO_ROOT / "tests" / "fixtures" / "social_offload" /
"core_artifact.md"


def _run_one(out_dir: Path, log_path: Path) ->
subprocess.CompletedProcess[str]:

    cmd = [

        sys.executable,

        str(SCRIPT),

        "--artifact", str(PASSING),

        "--channel", "linkedin",

        "--pack", str(PACK),

        "--out", str(out_dir),

        "--log", str(log_path),

    ]

    return subprocess.run(cmd, cwd=str(REPO_ROOT), text=True,
capture_output=True, check=False)


def test_concurrent_jsonl_append_is_not_corrupted(tmp_path: Path) -> None:

    out_dir = tmp_path / "outputs"

    log_path = tmp_path / "logs" / "social_offload_runs.jsonl"

    out_dir.mkdir(parents=True, exist_ok=True)
```

```
    log_path.parent.mkdir(parents=True, exist_ok=True)

    # Run multiple processes concurrently
    procs: list[subprocess.Popen[str]] = []
    N = 12
    for _ in range(N):
        cmd = [
            sys.executable,
            str(SCRIPT),
            "--artifact", str(PASSING),
            "--channel", "linkedin",
            "--pack", str(PACK),
            "--out", str(out_dir),
            "--log", str(log_path),
        ]
        procs.append(subprocess.Popen(cmd, cwd=str(REPO_ROOT), text=True,
stdout=subprocess.PIPE, stderr=subprocess.PIPE))

    results = [p.communicate() for p in procs]
    rcs = [p.returncode for p in procs]
    assert all(rc == 0 for rc in rcs), f"non-zero returncodes: {rcs},
stderr={results}"

    # Validate JSONL lines are complete JSON objects
    raw_lines = log_path.read_text(encoding="utf-8").splitlines()
    assert len(raw_lines) == N, f"expected {N} lines, got {len(raw_lines)}"

    parsed = []
    for i, line in enumerate(raw_lines):
```

```
        try:
            parsed.append(json.loads(line))
        except Exception as e:
            raise AssertionError(f"line {i} is not valid json: {line!r}") from e

    assert all(p.get("status") == "ok" for p in parsed)

    # Validate no duplicate render_id/run_id (depends on your fields; enforce at least one)
    run_ids = [p.get("run_id") for p in parsed]
    assert all(run_ids), "missing run_id in one or more records"
    assert len(set(run_ids)) == len(run_ids), "duplicate run_id detected"
```

`run_id`

`render_id`

`"ok"`

```
cd E:\signal_agent
```

```
.\.venv\Scripts\activate

python -m pytest -vv

python -m pytest -vv tests\integration\test_social_offload_concurrency.py
```

`*.tmp.*`

`social_offload.py` `main()`

Make sure to give me a prompt for codex High and then for anti-gravity to implement the above as we already specified in the picture I attached. So I want a codex high prompt and an anti-gravity prompt if need be to implement

```
You are Codex High. You will operate inside a local repo at:

E:\signal_agent

Mission:
Implement the “Take Advantage of What You Have” leverage stack item (1): harden dashboard, write test harness, remove path bugs, add logging introspection endpoint.

Non-negotiable constraints:
- Deterministic, single source of truth.
- Fail-closed behavior preserved (non-zero exit code on constraint violation, no output written).
- Logging contract must remain canonical:
 ROOT/data/social_offload/logs/social_offload_runs.jsonl
 with hermetic overrides in tests via --log.
- Windows-first paths (PowerShell + backslashes) but code should be path-robust via pathlib.
- Any changes must be backed by tests.
- Prefer small diffs; do not redesign architecture.

Work plan (execute in this order):

A) HARDEN DASHBOARD
```

1) Ensure HQ dashboard always passes explicit --out and --log to the subprocess runner for social_offload.

2) Make dashboard read Recent Runs ONLY from the same JSONL it writes to (no alternate paths).

3) Add a backend endpoint for logging introspection:

- GET /api/log_contract
- returns: canonical_log_path, exists(bool), size_bytes, last_modified_iso, last_n_lines (default 20, cap 200), and parsed last_n JSON records (skip malformed with an errors list)
- the endpoint MUST NOT crash if JSONL has malformed lines; report them.

B) WRITE TEST HARNESS (MECHANICAL CONTRACT TESTS)

4) Add/extend integration tests that assert:

- Passing fixture: exit code 0, output file exists, JSONL appended with {status:"ok"} and matching render_id/artifact_id.
- Failing fixture: exit code 1, NO new output created, JSONL appended with {status:"fault"} and fault codes present.

5) Add a hermetic harness wrapper used by tests:

- always uses tmp_path for --out and --log
- never touches ROOT/data/... during tests
- asserts no leakage by checking production JSONL mtime unchanged.

C) REMOVE PATH BUGS

6) Eliminate any sys.path insertion hacks in social_offload.py and related modules.

- Use proper package imports (or module execution) that work under:
  - python -m pytest
  - python app\agents\social_offload\social_offload.py ...

7) Normalize output path handling:

- All paths should be resolved via Path.resolve() at the boundary.

- When writing “expected output rel path” in logs, use forward-slash normalized relpaths for UI display.

D) ADD LOGGING INTROSPECTION ENDPOINT

8) Ensure /api/log_contract works with:

- default canonical log
- overridden --log paths used in dashboard or tests
- missing file case (exists=false, last_n empty).

Acceptance checks (must run and pass):

- python -m pytest -vv
- Run dashboard server, hit /api/log_contract in browser or curl, verify fields are correct.
- Manual smoke:
  - run passing artifact via CLI → ok log appended, output created
  - run failing artifact via CLI → fault log appended, no output created

Deliverables:

1) Provide a concise CHANGELOG in markdown (what changed and why).

2) List file diffs (paths) touched.

3) Provide exact commands I can run in PowerShell to verify (copy/paste).

4) Do not leave placeholders like <PACKAGE_NAME>. If a dependency is missing, add it to requirements in the correct place and install instructions.

Now begin. First, inspect existing code paths:

- app/agents/social_offload/social_offload.py
- app/hq/dashboard/hq_dashboard.py (or equivalent)
- tests/integration/test_social_offload_regression.py

```
Identify current log/out plumbing and then implement the plan.
```

```
You are Anti-Gravity. You are not here to patch code directly; you are
here to extend capability and enforce system coherence through artifacts:
specs, packs, invariants, and expansion plan.

Context:
Repo root: E:\signal_agent
We already unified a deterministic Social Offload Log Contract across CLI,
dashboard, and tests.

Mission:
Implement the "Take Advantage of What You Have" leverage stack items (2)
and (3):

2) Use Anti-Gravity to:
- Extend system capability
- Write new domain packs
- Expand channel coverage
- Improve artifact generation

3) Use the system itself to:
- Generate LinkedIn posts
- Offload
- Validate

Non-negotiable invariants (treat as law):
```

- Every execution produces: Validation → Structured Log → Exit Code → Artifact Side Effect

- Fail-closed: violations must never write output artifacts, but must write a fault record to JSONL.

- Packs are the governance unit: versioned, testable, and auditable.

- “Single source of truth” logging contract remains canonical; tests must remain hermetic.

Deliverables (produce these as concrete artifacts, not prose):

A) CAPABILITY EXTENSION SPEC (v1)

Create a markdown spec:

- app/specs/social_offload_capability_extension_v1.md

Include:

1) New capability targets (minimum 4) that are compatible with current system:

  - multi-channel routing (linkedin, x/twitter, reddit, facebook, substack placeholders ok)

  - templating expansion (tone variants, length variants, section policies)

  - lint/format stage before output commit

  - dry-run mode that logs planned outputs without writing generation files

2) For each capability: inputs, outputs, exit codes, log fields additions, and failure modes.

3) Define required new CLI flags (names + behavior) but keep backwards compatibility.

B) NEW DOMAIN PACKS (minimum 2)

Create two new packs under:

- constraints/packs/domain/

Each pack must include:

- explicit version

- hard limits (length, forbidden phrases, banned claims, etc.)

- deterministic rule IDs

- test fixtures (passing + failing) under tests/fixtures/<pack_name>/

- integration tests that assert fail-closed and correct fault_codes logging.

Suggested pack set (do not ask; pick and implement):

1) x_pack.yaml (short-form, strict character limits, hashtag rules)

2) substack_pack.yaml (long-form, section requirements, citations/links policy optional)

C) CHANNEL COVERAGE EXPANSION

Define a channel registry artifact:

- app/agents/social_offload/channel_registry_v1.yaml

It should map:

- channel → default pack

- channel → output directory naming scheme

- channel → template selection logic

- channel → max retries (default 0) and fail-closed behavior

This registry must be consumable by CLI and dashboard later.

D) ARTIFACT GENERATION IMPROVEMENTS

Produce a concrete template schema:

- app/agents/social_offload/templates/template_schema_v1.yaml

Define:

- required fields (title, hook, body, CTA, tags)

- optional fields (citations, disclaimers)

```
- normalization rules (whitespace, bullet formatting)

- deterministic render_id composition inputs

Also define:

- how a template is selected (rule-based) and how violations are surfaced
as fault_codes.


E) VERTICAL INTEGRATION RUNBOOK

Create a runbook:

- app/runbooks/vertical_integration_social_offload_v1.md

It must show the end-to-end loop:

1) generate post (from artifact input)

2) validate (constraints)

3) offload (write outputs)

4) validate again (contract checks)

Include copy/paste PowerShell commands and expected results.


Output format rules:

- Every deliverable must include exact file paths + exact contents.

- Prefer YAML/MD artifacts that can be dropped into the repo.

- Keep it deterministic: explicit versions, IDs, schemas.


Now generate the artifacts with full contents. No filler. No marketing.
```

Good. We'll harden two things:

1. atomic output write (no “ok” log line unless the file is already fully real)

2. concurrent-safe JSONL append (no interleaving / truncation / partial lines)

Below is the exact plan + paste-ready code blocks.

Step 0 — Target invariants

I1 (Atomic output): output file appears only via os.replace(tmp, final) after full write + fsync.

I2 (Log ordering): JSONL append happens after output commit for "ok", and after decision for "fault".

I3 (Concurrent append): multiple runs can append to the same JSONL without corrupting lines.

I4 (Crash-safe): if the process dies mid-write, you may get a stray *.tmp file, but never a half-written final output and never a partial JSON line.

Step 1 — Add a tiny util module (recommended)

Create: E:\signal_agent\app\utils\io_contract.py

```
# app/utils/io_contract.py
from __future__ import annotations

import json
import os
```

```
import random
import time
import uuid
from dataclasses import dataclass
from pathlib import Path
from typing import Any, Iterable, Optional

IS_WINDOWS = os.name == "nt"

if IS_WINDOWS:
    import msvcrt  # type: ignore
else:
    import fcntl  # type: ignore


@dataclass(frozen=True)
class AtomicWriteResult:
    final_path: Path
    tmp_path: Path


def ensure_parent_dir(path: Path) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)


def atomic_write_text(final_path: Path, text: str, encoding: str = "utf-8") ->
AtomicWriteResult:
    """
    Writes to final_path via temp file + fsync + atomic replace.
    Guarantees final_path is either old or fully new, never partial.
    """
    ensure_parent_dir(final_path)
    tmp_path = final_path.with_name(final_path.name + f".tmp.{uuid.uuid4().hex}")

    # Write temp
    with open(tmp_path, "w", encoding=encoding, newline="\n") as f:
        f.write(text)
        f.flush()
        os.fsync(f.fileno())
```

```
    # Atomic commit
    os.replace(tmp_path, final_path)
    return AtomicWriteResult(final_path=final_path, tmp_path=tmp_path)


class FileLock:
    """
    Cross-platform advisory lock using a dedicated lockfile.
    - Windows: msvcrt.locking
    - POSIX: fcntl.flock
    """
    def __init__(self, lock_path: Path) -> None:
        self.lock_path = lock_path
        self._fh = None

    def __enter__(self) -> "FileLock":
        ensure_parent_dir(self.lock_path)
        self._fh = open(self.lock_path, "a+b")
        if IS_WINDOWS:
            # Lock 1 byte from start; hold while open.
            self._fh.seek(0)
            msvcrt.locking(self._fh.fileno(), msvcrt.LK_LOCK, 1)
        else:
            fcntl.flock(self._fh.fileno(), fcntl.LOCK_EX)
        return self

    def __exit__(self, exc_type, exc, tb) -> None:
        assert self._fh is not None
        try:
            if IS_WINDOWS:
                self._fh.seek(0)
                msvcrt.locking(self._fh.fileno(), msvcrt.LK_UNLCK, 1)
            else:
                fcntl.flock(self._fh.fileno(), fcntl.LOCK_UN)
        finally:
            self._fh.close()
            self._fh = None


def append_jsonl_atomic(
```

```
    jsonl_path: Path,
    record: dict[str, Any],
    *,
    lock_path: Optional[Path] = None,
    retries: int = 30,
    base_sleep_s: float = 0.02,
) -> None:
    """
    Concurrent-safe JSONL append:
      - locks a lockfile
      - appends one line
      - flush + fsync
    """
    ensure_parent_dir(jsonl_path)
    if lock_path is None:
        lock_path = jsonl_path.with_suffix(jsonl_path.suffix + ".lock")

    payload = json.dumps(record, ensure_ascii=False) + "\n"

    # Simple retry loop for lock contention (esp. Windows + antivirus + slow IO)
    last_err: Optional[BaseException] = None
    for i in range(retries):
        try:
            with FileLock(lock_path):
                with open(jsonl_path, "a", encoding="utf-8", newline="\n") as f:
                    f.write(payload)
                    f.flush()
                    os.fsync(f.fileno())
            return
        except BaseException as e:
            last_err = e
            # Jittered backoff
            sleep = base_sleep_s * (1.2 ** i) + random.random() * base_sleep_s
            time.sleep(min(sleep, 0.5))
    raise RuntimeError(f"append_jsonl_atomic failed after {retries} retries: {last_err}")
from last_err


--
```

Step 2 — Refactor social_offload.py to enforce ordering

In your social_offload.py, use:

atomic_write_text() for outputs

append_jsonl_atomic() for JSONL

log "ok" only after output commit

ensure --log parent dir exists

add run_id (uuid) to every record

Minimum patch shape inside the success path:

build record dict (without status first if you want)

write output atomically

then append "ok" record

Minimum patch shape inside failure path:

append "fault" record (no output write)

If you want, paste the top ~80 lines and I'll give you an exact diff; but you can do this yourself cleanly with the util above.

Step 3 — Add a concurrency integration test

Create: tests/integration/test_social_offload_concurrency.py

This test intentionally runs many processes against the same --log.

```python
# tests/integration/test_social_offload_concurrency.py
from __future__ import annotations

import json
import os
import subprocess
import sys
from pathlib import Path

REPO_ROOT = Path(__file__).resolve().parents[2]
SCRIPT = REPO_ROOT / "app" / "agents" / "social_offload" / "social_offload.py"
PACK = REPO_ROOT / "constraints" / "packs" / "domain" / "linkedin_pack.yaml"
PASSING = REPO_ROOT / "tests" / "fixtures" / "social_offload" / "core_artifact.md"

def _run_one(out_dir: Path, log_path: Path) -> subprocess.CompletedProcess[str]:
    cmd = [
        sys.executable,
        str(SCRIPT),
        "--artifact", str(PASSING),
        "--channel", "linkedin",
        "--pack", str(PACK),
        "--out", str(out_dir),
        "--log", str(log_path),
    ]
    return subprocess.run(cmd, cwd=str(REPO_ROOT), text=True, capture_output=True,
check=False)

def test_concurrent_jsonl_append_is_not_corrupted(tmp_path: Path) -> None:
    out_dir = tmp_path / "outputs"
    log_path = tmp_path / "logs" / "social_offload_runs.jsonl"
    out_dir.mkdir(parents=True, exist_ok=True)
    log_path.parent.mkdir(parents=True, exist_ok=True)

    # Run multiple processes concurrently
    procs: list[subprocess.Popen[str]] = []
    N = 12
    for _ in range(N):
        cmd = [
            sys.executable,
```

```
        str(SCRIPT),
        "--artifact", str(PASSING),
        "--channel", "linkedin",
        "--pack", str(PACK),
        "--out", str(out_dir),
        "--log", str(log_path),
    ]
    procs.append(subprocess.Popen(cmd, cwd=str(REPO_ROOT), text=True,
stdout=subprocess.PIPE, stderr=subprocess.PIPE))

results = [p.communicate() for p in procs]
rcs = [p.returncode for p in procs]
assert all(rc == 0 for rc in rcs), f"non-zero returncodes: {rcs}, stderr={results}"

# Validate JSONL lines are complete JSON objects
raw_lines = log_path.read_text(encoding="utf-8").splitlines()
assert len(raw_lines) == N, f"expected {N} lines, got {len(raw_lines)}"

parsed = []
for i, line in enumerate(raw_lines):
    try:
        parsed.append(json.loads(line))
    except Exception as e:
        raise AssertionError(f"line {i} is not valid json: {line!r}") from e

assert all(p.get("status") == "ok" for p in parsed)

# Validate no duplicate render_id/run_id (depends on your fields; enforce at least
one)
run_ids = [p.get("run_id") for p in parsed]
assert all(run_ids), "missing run_id in one or more records"
assert len(set(run_ids)) == len(run_ids), "duplicate run_id detected"
```

If you don't have run_id yet, either add it (preferred) or change the uniqueness assertion to render_id.

Step 4 — Add one more invariant: "ok implies file exists"

You already do this conceptually. Now enforce it mechanically.

In your existing regression test (or add a new one), after reading the last "ok" record, assert:

expected output path exists

size > 0

(optional) output mtime <= log timestamp + small epsilon

Step 5 — Run commands

Paste-safe PowerShell run sequence

```
cd E:\signal_agent
.\.venv\Scripts\activate
python -m pytest -vv
python -m pytest -vv tests\integration\test_social_offload_concurrency.py
```

What You Should Expect When It's Correct

The concurrency test passes repeatedly (run it multiple times).

The JSONL file has exactly N lines, each line parses as JSON.

No partial line, no merged lines, no truncation.

Outputs exist for all successful runs (in their own render-id named paths).

One optional hardening (worth doing soon)

Add a cleanup on start:

delete *.tmp.* files older than, say, 24h in the output folder.

Not required for correctness, but keeps the workspace clean after a crash.

If you paste your current social_offload.py (imports through the end of main()), I'll give you a surgical diff that drops these helpers in with minimal movement and no behavior changes beyond the contract.

Those two prompts do this? I don't want you accidentally taking out of context the picture I sent because I didn't want the picture being implemented I wanted this plan being implemented through CLI prompting and anti-gravity prompting That's want to make sure we're on the same page here and that you didn't lose context of that when I sent the picture

```
You are Codex High. Repo root:

E:\signal_agent

Mission:
Implement two hardenings for Social Offload:
(1) atomic output write (never log status=ok unless the output file is fully committed)
(2) concurrent-safe JSONL append (no interleaving/truncation/partial lines under concurrent runs)

Treat the following invariants as testable laws:

I1 Atomic output:
- output written to tmp file
- flush + fsync
- commit via os.replace(tmp, final)
- final output is never partial

I2 Log ordering:
- For status="ok": JSONL append happens ONLY after output commit.
- For status="fault": JSONL append happens after the fail-closed decision (and no output is written).
```

I3 Concurrent append:

- multiple processes can append to the same JSONL safely without corrupting lines.

I4 Crash-safe:

- crash may leave *.tmp.* but never a half-written final output and never a partial JSON line.

Implementation plan (do in this order):

A) Add util module

1) Create app/utils/io_contract.py implementing:

- ensure_parent_dir(Path)

- atomic_write_text(final_path: Path, text: str) using tmp + fsync + os.replace

- append_jsonl_atomic(jsonl_path: Path, record: dict, lock_path optional) using a dedicated lockfile

 - Windows: msvcrt.locking

 - POSIX: fcntl.flock

 - lock contention retry with jitter

 - open file in append mode, write exactly one JSON line + "\n", flush + fsync

B) Refactor social_offload.py (minimal movement)

2) Integrate the util:

- add run_id=uuid4 hex to every record

- ensure --log parent dir exists

- enforce ordering:

 - PASS path: atomic_write_text(output_file, rendered_text) THEN append_jsonl_atomic(... status="ok")

- FAIL path: append_jsonl_atomic(... status="fault") and do not write output

- preserve exit codes: ok→0, fault→1

- preserve fail-closed behavior and existing log fields; only add run_id (and any needed path normalization)

C) Add concurrency integration test

3) Add tests/integration/test_social_offload_concurrency.py:

- spawn N subprocesses concurrently (Popen) all writing to SAME --log in tmp_path

- require all rc==0

- require JSONL has exactly N lines

- each line parses as JSON

- each has status=="ok"

- enforce uniqueness of run_id (or render_id if needed, but prefer run_id)

D) Add “ok implies file exists” invariant test

4) In an existing regression test or a new small test:

- read last ok record from JSONL

- compute expected output file path

- assert it exists and size>0

- (optional) mtime sanity check vs log timestamp

E) Verify

5) Run:

- python -m pytest -vv

- python -m pytest -vv tests\integration\test_social_offload_concurrency.py

```
Deliverables:

- Provide a concise CHANGELOG

- List files changed

- Provide copy/paste PowerShell verification commands


Do not:

- redesign architecture

- remove fail-closed constraints behavior

- introduce non-standard dependencies without putting them in the right
requirements location


Begin by inspecting current:

app/agents/social_offload/social_offload.py

tests/integration/test_social_offload_regression.py

then implement A→E.
```

```
You are Anti-Gravity. You do not patch code directly.

You produce governance artifacts that harden system coherence.


Repo root:

E:\signal_agent


Mission:

Turn these runtime invariants into explicit, versioned governance
artifacts that Codex High can implement and tests can enforce:


I1 Atomic output commit via tmp + fsync + os.replace

I2 Log ordering (ok only after commit; fault after decision)
```

I3 Concurrent-safe JSONL append (lockfile, single-line append, fsync)

I4 Crash-safe (tmp debris allowed, no partial final outputs, no partial JSON lines)

Deliverables (write full contents):

1) Spec:

app/specs/social_offload_io_contract_v1.md

Must include:

- the invariants as formal statements
- failure modes + expected observable behavior
- required log fields (including run_id)
- required ordering rules
- definition of “atomic output” and “append atomicity”

2) Test Contract:

app/specs/social_offload_test_contract_v1.md

Must include:

- concurrency test scenario requirements
- ok-implies-file-exists requirement
- hermetic test rules (tmp_path for --out/--log)
- prohibited leakage into canonical production log

3) Runbook:

app/runbooks/social_offload_io_hardening_verification_v1.md

Must include:

- copy/paste PowerShell commands for:
 - single-run ok/fault validation

```
 - concurrency test run
 - manual inspection (tail log, list outputs)
- expected outputs at each step


4) Optional cleanup policy:
app/specs/tmp_cleanup_policy_v1.md
- safe cleanup of *.tmp.* older than N hours
- explicitly non-required for correctness, but allowed for hygiene


Output the artifacts as file path + full content blocks that can be
dropped into the repo.
No marketing, no filler.
```

Social Offload - LinkedIn Revision

Implementation is complete and perfectly bounded. The deterministic pipeline successfully limits LinkedIn output to 250 words and fails closed on excess claims or forbidden phrasing.

1. Domain Pack (
constraints/packs/domain/linkedin_pack.yaml
)

```yaml
scope: "domain:linkedin"
pack_version: "1.1.0"
created_at: "2026-02-20T21:00:00Z"
boundary_conditions:
  max_word_count: 250
  min_word_count: 20
  max_claims: 4
disallowed_phrases:
  - "game changer"
  - "synergy"
  - "delve"
  - "crushing it"
  - "thought leader"
  - "🚀"
  - "🔥"
  - "link in bio"
  - "subscribe now"
  - "buy now"
allowed_output_classes: []
required_invariants: []
constraint_rules:
  - constraint_id: LINKEDIN_ALLOW_OFFLOAD
    rule_type: ALLOW
    trigger:
      capability_id: "content:text"
    predicate: {}
  - constraint_id: LINKEDIN_DENY_EXCESS_CLAIMS
    rule_type: DENY
    trigger:
      capability_id: "content:text"
    predicate:
      op: "GT"
      left: "snapshot.metrics.claims_count"
      right: 4
  - constraint_id: LINKEDIN_DENY_EXCESS_WORDS
```

```
    rule_type: DENY
    trigger:
      capability_id: "content:text"
    predicate:
      op: "GT"
      left: "snapshot.metrics.word_count"
      right: 250
  - constraint_id: LINKEDIN_DENY_SHORT_WORDS
    rule_type: DENY
    trigger:
      capability_id: "content:text"
    predicate:
      op: "LT"
      left: "snapshot.metrics.word_count"
      right: 20
```

2. Template (
app/agents/social_offload/templates/linkedin_post.jinja
)

jinja

```
{% set hook_text = sections.get('hook') or sections.get('preamble') or 'Untitled Update'
%}
{{ hook_text }}
{% if sections.get('body') %}
{{ sections.get('body') }}
{% else %}
{% for text in sections.values() %}
{% if loop.index > 1 %}{{ text }}
{% endif %}
{% endfor %}
{% endif %}
{% if claims %}
{% for claim in claims %}
• {{ claim }}
{% endfor %}
{% endif %}
{% if sections.get('conclusion') %}
{{ sections.get('conclusion') }}
{% endif %}
```

3. Test Fixtures

Passing Core Artifact (

tests/fixtures/social_offload/core_artifact.md
)
markdown
hook
Systems fail because complexity outpaces observability.
body
When building deterministic systems, the biggest risk isn't network failure. It's coherence drift.
We addressed this by formalizing robust constraint packs and decoupling generation from projection.
This ensures output stays rigidly bounded without needing a human in the loop.
key_takeaways
- Determinism requires hashes, not just IDs.
- Constraints must fail-closed.
- Observability is non-negotiable.
conclusion
Keep your systems rigid so your agents can be flexible.
Failing Core Artifact (
tests/fixtures/social_offload/core_artifact_failing.md
)
Fails due to 6 claims (LINKEDIN_DENY_EXCESS_CLAIMS) and forbidden phrasing like "synergy", "crushing it", and "🚀".

markdown
hook
This is a total game changer for the industry! 🚀
body
We are crushing it right now. Our synergy is off the charts.
This new platform will revolutionize the way we work. Delve into the details!
claims
- Claim 1
- Claim 2
- Claim 3
- Claim 4
- Claim 5
- Claim 6
4. Expected Results (Golden Snapshots)
Passing Run (Golden File Output)
File is deterministically hashed to:
data/social_offload/outputs/linkedin/5bc5a93df241/cbbd84128f7d.txt

text

Systems fail because complexity outpaces observability.

When building deterministic systems, the biggest risk isn't network failure. It's coherence drift.

We addressed this by formalizing robust constraint packs and decoupling generation from projection.

This ensures output stays rigidly bounded without needing a human in the loop.

- Determinism requires hashes, not just IDs.
- Constraints must fail-closed.
- Observability is non-negotiable.

Keep your systems rigid so your agents can be flexible.

JSONL Audit Append


```
{"timestamp": 1740003568.52, "artifact_path":
"E:\\signal_agent\\tests\\fixtures\\social_offload\\core_artifact.md", "artifact_id":
"5bc5a93df241", "render_id": "cbbd84128f7d", "channel": "linkedin", "pack_version":
"1.1.0", "template_version": "1.0.0", "status": "ok", "word_count": 68, "claim_count": 3}
```


Failing Run

Exit code 1, no output file written.


```
{"timestamp": 1740003571.39, "artifact_path":
"E:\\signal_agent\\tests\\fixtures\\social_offload\\core_artifact_failing.md", "artifact_id":
"d0eaafee4bad", "render_id": "bb65fa8962ed", "channel": "linkedin", "pack_version":
"1.1.0", "template_version": "1.0.0", "status": "fault", "fault_codes":
"['LINKEDIN_DENY_EXCESS_CLAIMS']"}
```


5. Integration Notes

Run the passing case: python app/agents/social_offload/social_offload.py --artifact tests/fixtures/social_offload/core_artifact.md --channel linkedin --pack constraints/packs/domain/linkedin_pack.yaml

Run the failing case: python app/agents/social_offload/social_offload.py --artifact tests/fixtures/social_offload/core_artifact_failing.md --channel linkedin --pack constraints/packs/domain/linkedin_pack.yaml

`1.1.0` `max_word_count` `max_claims`

`ok` `fault`

`run_id = uuid4()`
`"run_id": "<uuid>"`

`"fault_codes": "['LINKEDIN_DENY_EXCESS_CLAIMS']"`

`"fault_codes": ["LINKEDIN_DENY_EXCESS_CLAIMS"]`

`required_invariants: ["IO_CONTRACT_V1_ATOMIC_OUTPUT", "IO_CONTRACT_V1_JSONL_APPEND_LOCKED"]`

`os.replace()`

`Popen`

---

app/utils/io_contract.py
tests/integration/test_social_offload_concurrency.py

```
Patch requirements for log schema:

1) Add run_id to every JSONL record:
- import uuid
- run_id = uuid.uuid4().hex
- include "run_id": run_id in both ok and fault records

2) fault_codes MUST be a JSON array, not a string:
- Use: "fault_codes": ["CODE1", "CODE2"]
- Never: "fault_codes": "['CODE1']"
```

```
3) Keep existing keys stable (timestamp, artifact_path, artifact_id,
render_id, channel, pack_version, template_version, status, word_count,
claim_count)

Only add run_id and correct fault_codes typing.
```

`.txt`

`outputs/linkedin/<artifact>/<render>/...`

`.txt`

`ok`

`social_offload.py`

`run_id`

Okay, real quick, some context. What I copy and pasted was from Antigravity, and then the screenshot was from Codex.

```
ROLE: Codex High (implementation). Repo: E:\signal_agent.


MISSION:

Implement IO_CONTRACT_V1 for Social Offload:

- Atomic output write (no “ok” log entry unless output file is fully
committed).

- Concurrent-safe JSONL append (no interleaving/truncation/partial lines
under concurrent processes).

- Add tests proving this contract.


NON-GOALS:

Do NOT modify linkedin_pack.yaml constraints, templates, fixtures content,
or “golden output” wording except where tests require new fields (run_id)
or corrected typing (fault_codes as array).


INVARIANTS TO ENFORCE:
```

I1 Atomic output: write to temp file, flush+fsync, then os.replace(temp, final). Never partial final file.

I2 Log ordering: for status="ok", append JSONL only AFTER atomic output commit. For status="fault", append after decision, with no output write.

I3 Concurrent append: multiple parallel runs appending to same JSONL produce one valid JSON object per line, no corruption.

I4 Crash-safe: at worst stray *.tmp.* remains; never partial final output; never partial JSON line.

REQUIRED CHANGES:

1) Add app/utils/io_contract.py with:

- atomic_write_text(final_path, text) using temp + fsync + os.replace
- append_jsonl_atomic(jsonl_path, record) using lockfile (Windows msvcrt, POSIX fcntl), flush+fsync
- ensure_parent_dir

2) Refactor app/agents/social_offload/social_offload.py to use io_contract:

- Generate run_id = uuid4().hex per invocation and include in every JSONL record
- Ensure fault_codes is a JSON array (list[str]) not a stringified list
- Ensure --log parent dir is created (no assumptions)
- Keep existing record keys stable; only add run_id and fix fault_codes typing

3) Add tests:

- tests/integration/test_social_offload_concurrency.py

  Runs N parallel processes against same --log and asserts:

  a) all returncode==0 for passing fixture

  b) JSONL has exactly N lines

```
      c) each line parses as JSON

      d) run_id unique

  - Update/extend existing regression to assert:

      "ok implies output file exists and size>0"


DELIVERABLES:

- Provide exact diffs (or final file contents) for the new module,
refactored social_offload.py sections, and new tests.

- After edits, show the commands to run:

 python -m pytest -vv

 python -m pytest -vv tests\integration\test_social_offload_concurrency.py


QUALITY BAR:

- No behavior drift besides contract hardening.

- No pack/template "polish."

- Deterministic, minimal, auditable changes.
```

```
ROLE: Anti-Gravity (architecture/spec). Repo: E:\signal_agent.


CONTEXT:

Social Offload already has a unified log contract across CLI + HQ
Dashboard + tests. Now we harden IO under stress.


TASK:

Write IO_CONTRACT_V1 specification + acceptance criteria for "atomicity +
concurrency" hardening, suitable to hand to Codex High for implementation.


OUTPUT MUST INCLUDE:
```

```
1) Contract name + version: IO_CONTRACT_V1

2) Invariants I1–I4 (atomic output, log ordering, concurrent append, crash safety)

3) Exact record schema:

  - required fields

  - types (fault_codes MUST be list[str], run_id MUST be str)

4) Ordering rules (when ok/fault log append occurs relative to output)

5) Test matrix:

  - single-run passing

  - single-run failing

  - concurrent passing N-process append

  - negative tests (lock contention / missing parent dirs)

6) Operational implications:

  - lockfile location rules

  - tmp-file cleanup policy (optional)

7) "Done means" checklist:

  - which files exist

  - which tests pass

  - what repeated runs should look like


CONSTRAINTS:

- Do not rewrite LinkedIn pack/template rules unless the contract requires a schema field addition or type correction.

- Keep it bounded and mechanical; no narrative.
```

---

social_offload.py

Branched from LinkedIn Notification Count

```
import argparse
import hashlib
import json
import logging
import sys
import time
from pathlib import Path
from typing import Any

# Add project root to sys.path
sys.path.insert(0, str(Path(__file__).resolve().parent.parent.parent.parent))

from jinja2 import Environment, FileSystemLoader
import yaml

from app.utils.reprojection import reproject_checkpoint, extract_artifact_state
from app.utils.exceptions import ConstraintViolation
```

```
logging.basicConfig(level=logging.INFO, format="%(asctime)s [%(levelname)s]
%(message)s")
logger = logging.getLogger(__name__)

REPO_ROOT = Path(__file__).resolve().parents[3]
DEFAULT_OUTPUT_ROOT = REPO_ROOT / "data" / "social_offload" / "outputs"
DEFAULT_LOG_PATH = REPO_ROOT / "data" / "social_offload" / "logs" /
"social_offload_runs.jsonl"
TEMPLATE_VERSION = "1.0.0"

def sha256_hash(data: str) -> str:
    return hashlib.sha256(data.encode("utf-8")).hexdigest()


def _load_pack_version(pack_path: Path) -> str:
    try:
        with open(pack_path, "r", encoding="utf-8") as f:
            pack = yaml.safe_load(f) or {}
        value = pack.get("pack_version")
        if value is not None:
            return str(value)
    except Exception:
        pass
    return "unknown"


def _build_ids(artifact_path: Path, content: str | None, pack_version: str,
template_version: str) -> tuple[str, str]:
    artifact_basis = content if content is not None else str(artifact_path)
    artifact_id = sha256_hash(artifact_basis)[:12]
    render_id = sha256_hash(f"{artifact_id}_{pack_version}_{template_version}")[:12]
    return artifact_id, render_id


def _append_run_log(log_path: Path, log_entry: dict[str, Any]) -> None:
    log_path.parent.mkdir(parents=True, exist_ok=True)
    with open(log_path, "a", encoding="utf-8") as f:
        f.write(json.dumps(log_entry) + "\n")
```

```
def render_linkedin_output(state, template_dir: Path) -> str:
    env = Environment(loader=FileSystemLoader(str(template_dir)), trim_blocks=True,
lstrip_blocks=True)
    template = env.get_template("linkedin_post.jinja")
    rendered = template.render(sections=state.sections, claims=state.claims,
word_count=state.word_count)
    return rendered.strip()

def run_social_offload(artifact_path: Path, channel: str, out_dir: Path, pack_path: Path,
log_path: Path, dry_run: bool) -> int:
    artifact_path = artifact_path.resolve()
    pack_path = pack_path.resolve()
    out_dir = out_dir.resolve()
    log_path = log_path.resolve()

    pack_version = _load_pack_version(pack_path)
    template_version = TEMPLATE_VERSION
    artifact_id, render_id = _build_ids(artifact_path, content=None,
pack_version=pack_version, template_version=template_version)

    logger.info(f"Starting {channel} social offload for artifact {artifact_path}")

    try:
        with open(artifact_path, "r", encoding="utf-8") as f:
            content = f.read()
    except FileNotFoundError:
        logger.error(f"Artifact not found: {artifact_path}")
        write_fault_report(
            log_path=log_path,
            artifact_path=artifact_path,
            artifact_id=artifact_id,
            channel=channel,
            pack_version=pack_version,
            template_version=template_version,
            render_id=render_id,
            fault_details=f"Artifact not found: {artifact_path}",
        )
        return 1

    state = extract_artifact_state(content)
```

```
    artifact_id, render_id = _build_ids(artifact_path, content=content,
pack_version=pack_version, template_version=template_version)
    logger.info(f"Artifact ID: {artifact_id} | Render ID: {render_id}")

    # 1. Reproject constraint check
    try:
        reproject_checkpoint(
            artifact=content,
            pack_path=str(pack_path),
            execution_context_id=render_id,
            log_dir=log_path.parent / "reprojection",
        )
    except ConstraintViolation as e:
        logger.error(f"Failing closed due to constraint violation: {e}")
        write_fault_report(
            log_path=log_path,
            artifact_path=artifact_path,
            artifact_id=artifact_id,
            channel=channel,
            pack_version=pack_version,
            template_version=template_version,
            render_id=render_id,
            fault_details=str(e),
        )
        return 1

    # 2. Render Template
    template_dir = Path(__file__).resolve().parent / "templates"
    try:
        rendered_text = render_linkedin_output(state, template_dir)
    except Exception as e:
        logger.error(f"Template rendering failed: {e}")
        write_fault_report(
            log_path=log_path,
            artifact_path=artifact_path,
            artifact_id=artifact_id,
            channel=channel,
            pack_version=pack_version,
            template_version=template_version,
            render_id=render_id,
```

```
        fault_details=f"Template Error: {e}",
    )
    return 1

# 3. Write Output
if not dry_run:
    channel_out_dir = out_dir / channel / artifact_id
    channel_out_dir.mkdir(parents=True, exist_ok=True)
    out_file = channel_out_dir / f"{render_id}.txt"
    with open(out_file, "w", encoding="utf-8") as f:
        f.write(rendered_text)
    logger.info(f"Successfully wrote output to {out_file}")

# 4. Write Audit Log
log_entry = {
    "timestamp": time.time(),
    "artifact_path": str(artifact_path),
    "artifact_id": artifact_id,
    "render_id": render_id,
    "channel": channel,
    "pack_version": pack_version,
    "template_version": template_version,
    "status": "ok",
    "word_count": state.word_count,
    "claim_count": len(state.claims),
}
_append_run_log(log_path, log_entry)

logger.info("Pipeline completed successfully.")
return 0

def write_fault_report(
    log_path: Path,
    artifact_path: Path,
    artifact_id: str,
    channel: str,
    pack_version: str,
    template_version: str,
    render_id: str,
    fault_details: str,
```

```
) -> None:
    log_entry = {
        "timestamp": time.time(),
        "artifact_path": str(artifact_path),
        "artifact_id": artifact_id,
        "render_id": render_id,
        "channel": channel,
        "pack_version": pack_version,
        "template_version": template_version,
        "status": "fault",
        "fault_codes": fault_details,
    }
    _append_run_log(log_path, log_entry)

if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="Deterministic Social Offload")
    parser.add_argument("--artifact", required=True, type=Path, help="Core artifact path")
    parser.add_argument("--channel", required=True, choices=["linkedin"], help="Target
channel")
    parser.add_argument("--out", type=Path, default=DEFAULT_OUTPUT_ROOT,
help="Output root directory")
    parser.add_argument("--log", type=Path, default=DEFAULT_LOG_PATH,
help="Canonical JSONL log path")
    parser.add_argument("--pack", type=Path, required=True, help="Constraint pack
path")
    parser.add_argument("--dry-run", action="store_true", help="Do not write file")

    args = parser.parse_args()
    raise SystemExit(run_social_offload(args.artifact, args.channel, args.out, args.pack,
args.log, args.dry_run))
```

```
from uuid import uuid4
```

`from app.utils...`

```
from app.utils.io_contract import atomic_write_text, append_jsonl_atomic
```

`_append_run_log()`

```
def _append_run_log(log_path: Path, log_entry: dict[str, Any]) -> None:
    log_path.parent.mkdir(parents=True, exist_ok=True)
    with open(log_path, "a", encoding="utf-8") as f:
        f.write(json.dumps(log_entry) + "\n")
```

```
def _append_run_log(log_path: Path, log_entry: dict[str, Any]) -> None:
    append_jsonl_atomic(log_path, log_entry)
```

`run_id` `run_social_offload()`

`run_social_offload(...)`

```
artifact_path = artifact_path.resolve()
pack_path = pack_path.resolve()
out_dir = out_dir.resolve()
log_path = log_path.resolve()
```

```
run_id = uuid4().hex
```

`fault_codes` `list[str]`

`write_fault_report()`

```
"fault_codes": fault_details,
```

```
def write_fault_report(..., fault_details: str, ...) -> None:
```

```
def write_fault_report(..., fault_codes: list[str] | str, run_id: str,
...) -> None:
```

```
if isinstance(fault_codes, str):
   fault_list = [fault_codes]
else:
   fault_list = fault_codes
```

```
"fault_codes": fault_list,
"run_id": run_id,
```

`run_id`

`"ok"`

```
with open(out_file, "w", encoding="utf-8") as f:
    f.write(rendered_text)
```

```
atomic_write_text(out_file, rendered_text)
```

`write_fault_report()`

```
def write_fault_report(
    log_path: Path,
    artifact_path: Path,
    artifact_id: str,
    channel: str,
    pack_version: str,
    template_version: str,
    render_id: str,
    run_id: str,
    fault_codes: list[str] | str,
) -> None:
    if isinstance(fault_codes, str):
        fault_list = [fault_codes]
    else:
        fault_list = fault_codes

    log_entry = {
```

```
        "timestamp": time.time(),
        "artifact_path": str(artifact_path),
        "artifact_id": artifact_id,
        "render_id": render_id,
        "run_id": run_id,
        "channel": channel,
        "pack_version": pack_version,
        "template_version": template_version,
        "status": "fault",
        "fault_codes": fault_list,
    }
    _append_run_log(log_path, log_entry)
```

```
# 3. Write Output (ATOMIC)
out_file: Path | None = None
if not dry_run:
    channel_out_dir = out_dir / channel / artifact_id
    channel_out_dir.mkdir(parents=True, exist_ok=True)
    out_file = channel_out_dir / f"{render_id}.txt"
    atomic_write_text(out_file, rendered_text)
    logger.info(f"Successfully wrote output to {out_file}")

# 4. Write Audit Log (AFTER output commit)
log_entry = {
    "timestamp": time.time(),
    "artifact_path": str(artifact_path),
```

```
    "artifact_id": artifact_id,
    "render_id": render_id,
    "run_id": run_id,
    "channel": channel,
    "pack_version": pack_version,
    "template_version": template_version,
    "status": "ok",
    "word_count": state.word_count,
    "claim_count": len(state.claims),
}
_append_run_log(log_path, log_entry)
```

`run_id`

`write_fault_report(... fault_details=...)`

```
write_fault_report(
    log_path=log_path,
    artifact_path=artifact_path,
    artifact_id=artifact_id,
    channel=channel,
    pack_version=pack_version,
    template_version=template_version,
    render_id=render_id,
    run_id=run_id,
    fault_codes=str(e),    # or ["CODE"] if you have real codes
)
```

`app/utils/io_contract.py`

`app/utils/io_contract.py`

`_append_run_log()` `json.dumps(log_entry)` `ensure_ascii=False`

`app/utils/io_contract.py`

ok give me the cli and antigravity pro mpts to implament this

```
python -m pytest   Remove-Item
```

Implementing IO_CONTRACT_V1
Created io_contract.py with atomic file replacement and concurrent JSONL appending. Refactored social_offload.py to adopt these interfaces explicitly executing OK logging post-output and preventing fault logs from triggering outputs. Built and stabilized the standalone concurrency verification test validating 12 concurrent workers seamlessly executing against lock contention.

Files Edited
Progress Updates
1
Creating io_contract.py and updating task list

2
Updating social offload execution script to use IO Contract tools

3
Creating concurrency integration tests

4
Running the concurrency integration test

5
Running concurrency test via standard Python

6
Bulletproofing the atomic write against Windows locking

7
Notifying user of IO Contract V1 implementation

The IO_CONTRACT_V1 implementation and physical validation are complete.

Action Summary:

Added app/utils/io_contract.py to standardize multi-process atomic IO. atomic_write_text utilizes temporary siblings followed by flush, fsync, and a Windows-resilient atomic os.replace. append_jsonl_atomic achieves collision-free stream logging using low-level explicit O_CREAT | O_EXCL | O_WRONLY short-lived lock files natively compatible across OS boundaries.
Updated app/agents/social_offload/social_offload.py to use the IO tools natively: Each execution now hashes a unique run_id to strictly trace parallel calls sharing an identical artifact_id and render_id.
Output writes execute atomically.
The log stream exclusively appends status: "ok" only after the file has successfully dropped to disk and its size is inherently validated as non-empty.
Created tests/integration/test_social_offload_concurrency.py simulating 12 tightly packed concurrent subprocess workers writing to the same disk and JSONL file. 12 flawless writes were recorded sequentially without thread collision. Tests run to completely clean parity.
The execution environment is now deterministic, parallel-safe, and natively resilient to partial-file corruption across sudden termination instances.

this is what anti gravity said and did

```
O_CREAT | O_EXCL | O_WRONLY
```

```
msvcrt.locking
```

`app/utils/io_contract.py`

`atomic_write_text(...)`

`flush()` `os.fsync(fd)`
`os.replace(tmp, final)`

`append_jsonl_atomic(...)`

`msvcrt.locking`
`O_EXCL`

`json.dumps(record) + "\n"`
`flush()` `os.fsync(fd)`

`O_EXCL`

`finally`

`social_offload.py`

`run_id = uuid4().hex`
`status="ok"` `atomic_write_text`

`fault_codes`

`subprocess.Popen`

`--log`

📦

```
cd E:\signal_agent
.\.venv\Scripts\activate

# 1) Run the concurrency test repeatedly (stress)
python -m pytest -vv tests\integration\test_social_offload_concurrency.py
python -m pytest -vv tests\integration\test_social_offload_concurrency.py
python -m pytest -vv tests\integration\test_social_offload_concurrency.py

# 2) Validate the test log is not corrupt (quick parse check)
python -c "import json,sys; from pathlib import Path; import glob;
p=list(Path('E:/signal_agent').rglob('test_social_offload_concurrency.py')
)[0].parent;
print('testfile=',p); sys.exit(0)"
```

`-s`

`app/utils/io_contract.py`
`tests/integration/test_social_offload_concurrency.py`

`io_contract.py`

`os.replace`
`os.fsync`
`O_EXCL` `msvcrt.locking`
`finally:`

`social_offload.py`

`run_id`
`"status": "ok"`
`atomic_write_text`
`fault_codes`

---

can you give me a cli prompt to check all that stuff please

```
You are Codex High. Repo: E:\signal_agent.

Task: Perform a mechanical verification audit that IO_CONTRACT_V1 is truly
implemented (not just described), across:
- app/utils/io_contract.py
- app/agents/social_offload/social_offload.py
- tests/integration/test_social_offload_concurrency.py

Do NOT implement new features unless a contract violation is found. This
is an audit-first pass.

Audit requirements (must be answered with evidence from code + test runs):

A) io_contract.py correctness
1) Confirm atomic_write_text exists and enforces:
  - temp sibling file in same directory as final
  - write + flush + os.fsync
  - commit via os.replace(tmp, final)
  Provide exact line references or code excerpts (<=25 words per excerpt).
2) Confirm append_jsonl_atomic exists and enforces:
  - exclusive lock acquisition (either msvcrt.locking/fcntl OR O_EXCL
lockfile)
  - retry/backoff for contention
  - JSON line append with newline
```

```
  - flush + os.fsync

  - lock release + cleanup in finally

  Provide evidence as above.


B) social_offload.py ordering + schema

3) Confirm run_id is generated once per run (uuid4().hex) and included in
BOTH ok and fault log records.

4) Confirm log ordering:

  - ok log append happens only AFTER output commit (atomic_write_text)

  - fault log append happens without any output write

5) Confirm fault_codes is typed as list[str] in JSON (not stringified
list).


C) concurrency test validity

6) Confirm test spawns N=12 subprocesses writing to the SAME --log path
(tmp_path) concurrently.

7) Confirm test asserts:

  - all return codes == 0

  - JSONL has exactly N lines

  - each line parses as JSON

  - status=="ok"

  - run_id exists and is unique

8) Ensure test is hermetic: no writes to production logs under
ROOT/data/social_offload/logs/.


Execution steps (must run and show results):

- python -m pytest -vv
tests/integration/test_social_offload_concurrency.py

- python -m pytest -vv (full suite)
```

```
If ANY audit item fails:
- fix the minimal code required to satisfy IO_CONTRACT_V1
- rerun the same tests
- report what changed (file + summary + why)

Output format:
1) Audit report (A/B/C), each item marked PASS/FAIL with brief evidence.
2) If changes made: list files changed + minimal diff summary.
3) Commands run + pass/fail summary.

Begin by opening the three files, then execute the tests, then produce the report.
```